

1.3 Object Orientation

Object orientation

If we want to model in an object-oriented style, we must first clarify what *object orientation* means. The introduction of object orientation dates back to the 1960s when the simulation language SIMULA [24] was presented, building on a paradigm that was as natural to humans as possible to describe the world. The object-oriented approach corresponds to the way we look at the real world; we see it as a society of autonomous individuals, referred to as objects, which take a fixed place in this society and must thereby fulfill predefined obligations.

There is not only one single definition for object orientation. However, there is a general consensus about the properties that characterize object orientation. Naturally, objects play a central role in object-oriented approaches. Viewed simply, objects are elements in a system whose data and operations are described. Objects interact and communicate with one another. In general, we expect the concepts described below from an object-oriented approach.

1.3.1 *Classes*

Class

In many object-oriented approaches, it is possible to define *classes* that describe the attributes and the behavior of a set of objects (the instances of a class) abstractly and thus group common features of objects. For example, people have a name, an address, and a social security number. Courses have a unique identifier, a title, and a description. Lecture halls have a name as well as a location, etc. A class also defines a set of permitted operations that can be applied to the instances of the class. For example, you can reserve a lecture hall for a certain date, a student can register for an exam, etc. In this way, classes describe the behavior of objects.

1.3.2 *Objects*

Object

The instances of a class are referred to as its *objects*. For example, `lh1`, the Lecture Hall 1 of the Vienna University of Technology, is a concrete instance of the class `LectureHall`. In particular, an object is distinguished by the fact that it has its own identity, that is, different instances of a class can be uniquely identified. For example, the beamer in Lecture Hall 1 is a different object to the beamer in Lecture Hall 2, even

if the devices are of the same type. Here we refer to *identical* devices but not the *same* device. The situation for concrete values of data types is different: the number 1, which is a concrete value of the data type `Integer`, does not have a distinguishable identity.

An object always has a certain state. A state is expressed by the values of its attributes. For example, a lecture hall can have the state *occupied* or *free*. An object also displays behavior. The behavior of an object is described by the set of its operations. Operations are triggered by sending a message.

1.3.3 Encapsulation

Encapsulation is the protection against unauthorized access to the internal state of an object via a uniquely defined interface. Different levels of visibility of the interfaces help to define different access authorizations. Java, for example, has the explicit visibility markers `public`, `private`, and `protected`, which respectively permit access for all, only within the object, and only for members of the same class, its subclasses, and of the same package.

Encapsulation

1.3.4 Messages

Objects communicate with one another through *messages*. A message to an object represents a request to execute an operation. The object itself decides whether and how to execute this operation. The operation is only executed if the sender is authorized to call the operation—this can be regulated in the form of visibilities (see the previous paragraph)—and a suitable implementation is available. In many object-oriented programming and modeling languages the concept of *overloading* is supported. This enables an operation to be defined differently for different types of parameters. For example, the operator `+` realizes different behavior depending on whether it is used to add up integers (e.g., `1 + 1 = 2`) or to concatenate character strings (e.g., `"a" + "b" = "ab"`).

Message

Overloading

1.3.5 Inheritance

The concept of *inheritance* is a mechanism for deriving new classes from existing classes. A subclass derived from an existing class (= su-

Inheritance

perclass) inherits all visible attributes and operations (specification and implementation) of the superclass. A subclass can:

- Define new attributes and/or operations
- Overwrite the implementation of inherited operations
- Add its own code to inherited operations

Class hierarchy

Inheritance enables extensible classes and as a consequence, the creation of *class hierarchies* as the basis for object-oriented system development. A class hierarchy consists of classes with similar properties, for example, $\text{Person} \leftarrow \text{Employee} \leftarrow \text{Professor} \leftarrow \dots$ where $A \leftarrow B$ means that B is a subclass of A.

When used correctly, inheritance offers many advantages: reuse of program or model parts (thus avoiding redundancy and errors), consistent definition of interfaces, use as a modeling aid through a natural categorization of the occurring elements, and support for incremental development, i.e., a step-by-step refinement of general concepts to specific concepts.

1.3.6 Polymorphism

Polymorphism

In general terms, *polymorphism* is the ability to adopt different forms. During the execution of a program, a polymorphic attribute can have references to objects from different classes. When this attribute is declared, a type (e.g., class `Person`) is assigned statically at compile time. At runtime, this attribute can also be bound dynamically to a subtype (e.g., subclass `Employee` or subclass `Student`).

A polymorphic operation can be executed on objects from different classes and have different semantics in each case. This scenario can be implemented in many ways: (i) via *parametric polymorphism*, better known as genericity—here, type parameters are used. In Java for example, the concrete classes are transferred to the operations as arguments; (ii) via *inclusion polymorphism*—operations can be applied to classes and to their direct and indirect subclasses; (iii) via *overloading of operations*; and (iv) via *coercion*, that is, the conversion of types. The first two methods above are known as *universal polymorphism*; the other two methods are referred to as *ad hoc polymorphism* [13].

1.4 The Structure of the Book

In Chapter 2 we give a short overview of UML by recapitulating the history of its creation and taking a brief look at its 14 different diagrams. Then, in Chapter 3, we introduce the concepts of the use case diagram. This diagram enables us to describe the requirements that a system to be developed should satisfy. In Chapter 4 we present the class diagram. This diagram allows us to describe the structure of a system. To enable us to model the behavior of a system, in Chapter 5 we introduce the state machine diagram, in Chapter 6 the sequence diagram, and in Chapter 7 the activity diagram. We explain the interaction of the different types of diagrams in Chapter 8 with three examples. In Chapter 9, we briefly examine advanced concepts that are of significant importance for the practical use of UML.

The concepts are all explained using examples, all of which are based on the typical Austrian university environment. In most cases they represent heavily simplified scenarios. It is not our intention in this book to model one single, continuous system, as there is a high risk that in doing so we would become lost in a multitude of technical details. We have therefore selected examples according to their didactic benefit and their illustrative strength of expression. In many cases, we have therefore made assumptions that, for didactic reasons, are based on simplified presentations of reality.

UML is based entirely on object-oriented concepts. This is particularly noticeable in the class diagram, which can easily be translated into an object-oriented programming language. We will get to know the class diagram and possible translations to program code in Chapter 4. However, UML has not been designed for one specific object-oriented language. For the sake of readability, we use a notion of object-orientation as found in modern programming languages like Java or C#.

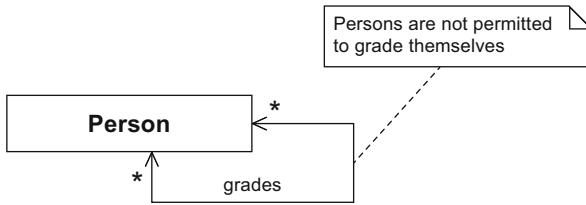


Figure 2.3
Example of a note

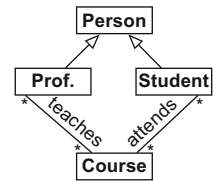
2.3.1 Structure Diagrams

UML offers seven types of diagrams for modeling the structure of a system from different perspectives. The dynamic behavior of the elements in question (i.e., their changes over time) is not considered in these diagrams.

The Class Diagram

Just like the concepts of the object diagram (see next paragraph), the concepts of the *class diagram* originate from conceptual data modeling and object-oriented software development. These concepts are used to specify the data structures and object structures of a system. The class diagram is based primarily on the concepts of *class*, *generalization*, and *association*. For example, in a class diagram, you can model that the classes *Course*, *Student*, and *Professor* occur in a system. Professors teach courses and students attend courses. Students and professors have common properties as they are both members of the class *Person*. This is expressed by a generalization relationship.

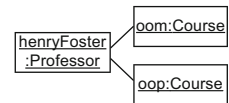
Class diagram
(see Chapter 4)



The Object Diagram

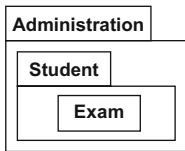
Based on the definitions of the related class diagram, an *object diagram* shows a concrete snapshot of the system state at a specific execution time. For example, an object diagram could show that a professor Henry Foster (henryFoster) teaches the courses Object-Oriented Modeling (oom) and Object-Oriented Programming (oop).

Object diagram
(see Chapter 4)



The Package Diagram

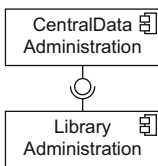
Package diagram



The *package diagram* groups diagrams or model elements according to common properties, such as functional cohesion. For example, in a university administration system, you could introduce packages that contain information about the teaching, the research, and the administrative aspects. Packages are often integrated in other diagrams rather than being shown in separate diagrams.

The Component Diagram

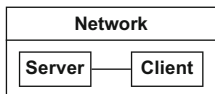
Component diagram



UML pays homage to component-oriented software development by offering *component diagrams*. A component is an independent, executable unit that provides other components with services or uses the services of other components. UML does not prescribe any strict separation between object-oriented and component-oriented concepts. Indeed, these concepts may be combined in any way required. When specifying a component, you can model two views explicitly: the external view (black box view), which represents the specification of the component, and the internal view (white box view), which defines the implementation of the component.

The Composition Structure Diagram

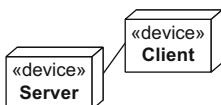
Composition structure diagram



The *composition structure diagram* allows a hierarchical decomposition of the parts of the system. You can therefore use a composition structure diagram to describe the internal structure of classes or components in detail. This enables you to achieve a higher level of detail than, for example, in a class diagram because the modeling is context-specific. You can specify details of the internal structure that are valid precisely for the context under consideration.

The Deployment Diagram

Deployment diagram

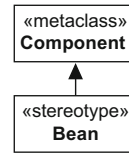


The hardware topology used and the runtime system assigned can be represented by the *deployment diagram*. The hardware encompasses processing units in the form of nodes as well as communication relationships between the nodes. A runtime system contains artifacts that are deployed to the nodes.

The Profile Diagram

Using *profiles*, you can extend UML to introduce domain-specific concepts. The actual core of the language definition of UML, the meta-model, remains unchanged. You can thus reuse modeling tools without having to make adjustments. For example, you can use profiles to introduce the concept of Java Enterprise Beans.

Profile diagram



2.3.2 Behavior Diagrams

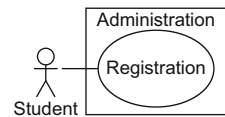
With the *behavior diagrams*, UML offers the infrastructure that enables you to define behavior in detail.

Behavior refers to the direct consequences of an action of at least one object. It affects how the states of objects change over time. Behavior can either be specified through the actions of a single object or result from interactions between multiple objects.

The Use Case Diagram

UML offers the *use case diagram* to enable you to define the requirements that a system must fulfill. This diagram describes which users use which functionalities of the system but does not address specific details of the implementation. The units of functionality that the system provides for its users are called *use cases*. In a university administration system, for example, the functionality Registration would be a use case used by students.

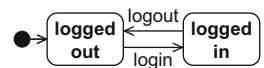
Use case diagram
(see Chapter 3)



The State Machine Diagram

Within their life cycle, objects go through different states. For example, a person is in the state logged out when first visiting a website. The state changes to logged in after the person successfully entered username and password (event login). As soon as the person logs out (event logout), the person returns to the state logged out. This behavior can be represented in UML using the *state machine diagram*. This diagram describes the permissible behavior of an object in the form of possible states and state transitions triggered by various events.

State machine diagram
(see Chapter 5)



Chapter 4

The Class Diagram

We use the *class diagram* to model the static structure of a system, thus describing the elements of the system and the relationships between them. These elements and the relationships between them do not change over time. For example, students have a name and a matriculation number and attend various courses. This sentence covers a small part of the university structure and does not lose any validity even over years. It is only the specific students and courses that change.

Class diagram

The class diagram is without doubt the most widely used UML diagram. It is applied in various phases of the software development process. The level of detail or abstraction of the class diagram is different in each phase. In the early project phases, a class diagram allows you to create a conceptual view of the system and to define the vocabulary to be used. You can then refine this vocabulary into a programming language up to the point of implementation. In the context of object-oriented programming, the class diagram visualizes the classes a software system consists of and the relationships between these classes. Due to its simplicity and its popularity, the class diagram is ideally suited for quick sketches. However, you can also use it to generate program code automatically. In practice, the class diagram is also often used for documentation purposes.

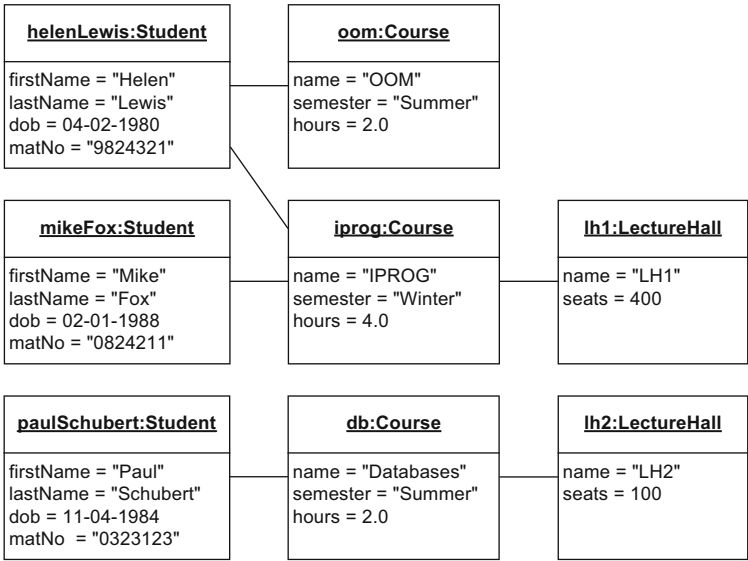
Before we introduce the concepts of the class diagram, let us first take a look at *objects*, which are modeled in *object diagrams*. Object diagrams allow you to depict concrete objects that appear in a system at a specific point in time. Classes provide schemas for characterizing objects and objects are instances of classes. The object diagram visualizes instances of classes that are modeled in a class diagram.

Object diagram

4.1 Objects

A system contains numerous different individuals. Individuals might be not only persons but also animals, plants, inanimate objects, artifacts, etc. that can be identified uniquely. For example, as part of her *IT Studies* program, *Helen Lewis* attends the lecture *Object-Oriented Modeling (OOM)* at the university. *Helen Lewis*, *IT Studies*, and *Object-Oriented Modeling* are individuals (concrete objects) in a university administration system and are in a relationship with one another.

Figure 4.1
Example of an object diagram



In UML, we depict concrete *objects* of a system and their relationships (*links*) using *object diagrams*. [Figure 4.1](#) shows a small object diagram. It contains three student objects: *helenLewis*, *mikeFox*, and *paulSchubert*. The first name and the last name of the object *helenLewis* are *Helen* and *Lewis* respectively. We also know the date of birth and matriculation number for each of these objects. The system contains the three courses *oom* (Object-Oriented Modeling), *iprog* (Introduction to Programming), and *db* (Databases). The course *iprog* takes place in lecture hall *lh1* and the course *db* takes place in lecture hall *lh2*. There is no corresponding information for *oom*. Student *helenLewis* attends the two courses *oom* and *iprog*. Student *mikeFox* also attends *iprog*; course *db* is attended only by student *paulSchubert* (at least, among these three students).

An object has a unique identity and a number of characteristics that describe it in more detail. It rarely appears in isolation in a system; instead, it usually interacts and communicates with other objects. The relationships between the objects are referred to as *links*. The characteristics of an object include its *structural characteristics* (attributes) and its *behavior* (in the form of operations). Whilst concrete values are assigned to the attributes in the object diagram, operations are generally not depicted. Operations are identical for all objects of a class and are therefore usually described exclusively for the class.

In the object diagram, an object is shown as a rectangle which can be subdivided into multiple compartments. The first compartment always contains information in the form `objectName:Class`. This information is centered and underlined. In [Figure 4.1](#) for example, `helenLewis` and `oom` are object names and `Student` and `Course` are classes. The object name or the specification of the class may be omitted. If only a class name is given, it must be preceded by a colon. If the class name is omitted, the colon is also omitted. If the object name is omitted, this object is referred to as an *anonymous object*. Examples of different notation alternatives are shown in [Figure 4.2](#).

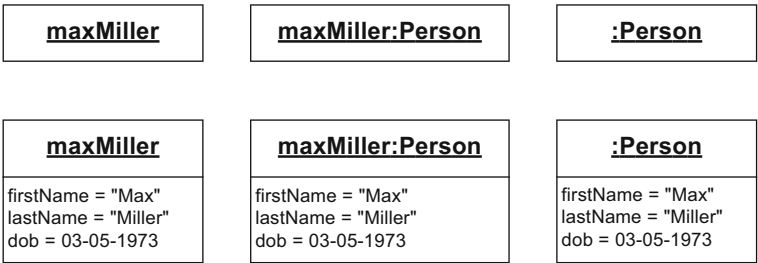


Figure 4.2
Notation alternatives for objects

If the rectangle has a second compartment, this compartment contains the attributes of the object and the current values of these attributes (see [Fig. 4.1](#) and [Fig. 4.2](#)). A link is represented as a continuous line connecting the objects that are in a relationship with one another. Although the name of an object must be unique, different objects can have attributes with identical values. If, in our system, there were two people with the first name *Max* and the last name *Miller*, and both were born on the same day, we would have to represent them using different objects with different object names (e.g., `maxMiller1` and `maxMiller2`). However, their attribute values would be identical.

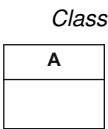
The values of the attributes generally change over time. For example, if the person *Max Miller* changes his last name, the individual as a whole does not change, only the value of the attribute `lastName`. The ob-

ject diagram therefore always represents only a snapshot of objects at a specific moment in time and the objects can develop further and change as time passes. If specific objects are not represented in the object diagram, this does not mean that they do not exist; it merely expresses that the unrecorded objects are not important for the moment.

From object to class

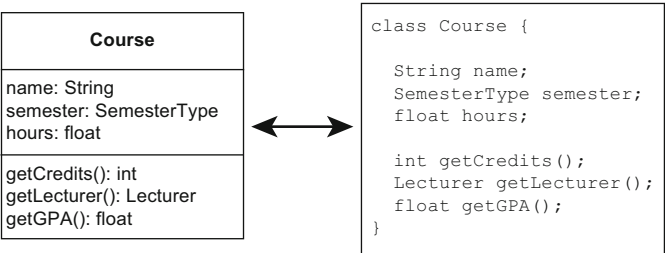
Many individuals that appear in a system have identical characteristics and behavior. For example, persons always have a first name, a last name, and a date of birth. Students also have a matriculation number. Courses always have a name and a number of hours, as well as a semester in which they take place. Information about the lecture halls includes the number of seats available. If every person, every course, and every lecture hall of the system were to be modeled individually, the model would soon become over-complicated and impossible to maintain. Using classes enables you to describe similar objects without having to detail each and every object individually.

4.2 Classes



A *class* is the construction plan for a set of similar *objects* that appear in the system to be specified. Classes can characterize, for example, persons (e.g., students), things (e.g., buildings), events (e.g., courses or exams), or even abstract concepts such as groups. In object-oriented programming languages like Java [4], programs are created based on classes. [Figure 4.3](#) compares a class definition from a UML class diagram with a class definition in Java.

Figure 4.3
Definition of a class in
UML and Java



Instance
Characteristics of
classes ...

Objects represent the concrete forms of classes and are referred to as their *instances*. The relevant characteristics of the instances of a class are described through the definition of structural characteristics (*attributes*) and behavior (*operations*). Operations enable objects to communicate with one another and to act and react.

An attribute allows you to store information that is known for all instances but that generally has different specific values for each instance. Operations specify how specific behavior can be triggered on individual objects. For example, the class `Course` from Figure 4.3 has the attributes `name` and `hours`. Figure 4.1 shows concrete forms of these attributes. Possible operations of this class are `getGPA()` and `getLecturer()`, which return the grade point average or lecturer for a course respectively.

To ensure that a model remains clear and understandable, we generally do not model all of the details of the content: we only include the information that is relevant for the moment and for the system to be implemented. This means that we *abstract* from reality to make the model less complex and to avoid an unnecessary flood of information. In the model, we restrict ourselves to the essentials. For example, in a university administration system, it is important to be able to manage the names and matriculation numbers of the students; in contrast, their shoe size is irrelevant and is therefore not included.

... are attributes and operations

Level of detail

Abstraction

4.2.1 Notation

In a class diagram, a class is represented by a rectangle that can be subdivided into multiple compartments. The first compartment must contain the name of the class, which generally begins with a capital letter and is positioned centered in bold font (e.g., `Course` in Figure 4.4).

According to common naming conventions, class names are singular nouns. The class name should describe the class using vocabulary typical for the application domain. The second compartment of the rectangle contains the *attributes* of the class, and the third compartment the

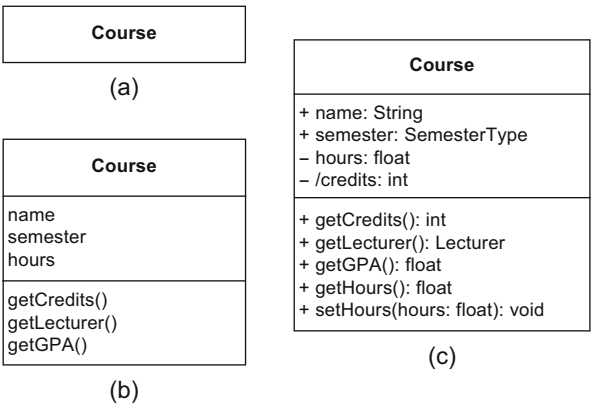


Figure 4.4
Representation of a class
and its characteristics

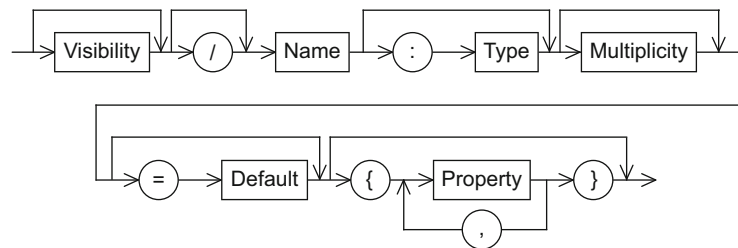
operations of the class. The contents of these compartments are positioned left-justified and are optional. In general, the level of detail in these compartments reflects the respective phase of the software development process in which the class is being examined. While the class diagram excerpt in Figure 4.4(a) does not contain any details of the class `Course`, the diagram in Figure 4.4(b) is the result of a more detailed analysis of the characteristic features of courses, showing specifically that the class `Course` contains three attributes and three operations. The diagram in Figure 4.4(c) presents even more detail (such as the type information and visibilities), including information that is relevant for implementation or for automatic code generation. If specific information is not included in the diagram, this does not mean that it does not exist; it simply means that this information is not relevant at this moment in time or is not included for practical reasons, for example, to prevent the diagram from becoming over-complicated. Attributes and operations are usually accessed via their names, which, according to naming conventions, begin with a lower case letter.

4.2.2 Attributes

Attribute Figure 4.5 shows the syntax of attributes. An attribute has at least a name. The type of the attribute may be specified after the name using `: Type`. Possible attribute types include primitive data types, such as integer and string, composite data types, for example a date, an enumeration, or user-defined classes (see Section 4.8). By specifying `name: String`, for example, we define the attribute name with type `String`. Figure 4.6 shows further examples of attribute types. We will look at the subsequent, optional multiplicity specification in more detail in the next section.

Type

Figure 4.5
Syntax of the attribute
specification



To define a *default value* for an attribute, you specify `= Default`, where *Default* is a user-defined value or expression (see Fig. 4.6). The system uses the default value if the value of the attribute is not set explicitly by the user. Thus it is impossible that at some point in time, an attribute has no value. For example, if in our system, a person must always have a password, a default password *pw123* is set when a new person is entered in the system. This password is valid until it is reseted.

Default value

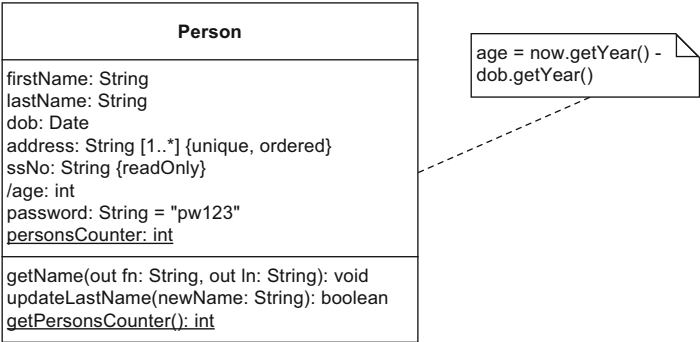


Figure 4.6
Properties of attributes

You can specify additional *properties* of the attribute within curly brackets. For example, the property `{readOnly}` means that the attribute value cannot be changed once it has been initialized. In the example in Figure 4.6, the social security number `ssNo` is an attribute that must not be changed. Further properties will be introduced in the next section within the description of multiplicity specifications.

Properties of attributes

The specification of a forward slash before an attribute name indicates that the value of this attribute is derived from other attributes. An example of a *derived attribute* is a person’s age, which can be calculated from the date of birth. In Figure 4.6, a note contains a calculation rule for determining a person’s age. Depending on the development tool used, such notes are formulated in natural language, in a programming language, or in pseudocode. The optional visibility marker (`+`, `-`, `#`, or `~`) in front of an attribute name or operation name as shown in Figure 4.4(c) is discussed in detail on page 58.

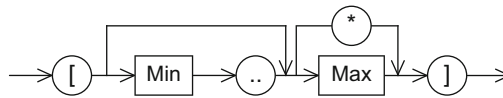
4.2.3 Multiplicities

The *multiplicity* of an attribute indicates how many values an attribute can contain. This enables you to define arrays, just like in programming languages. The multiplicity is shown as an interval enclosed by square

Multiplicity

brackets in the form [minimum .. maximum], whereby minimum and maximum are natural numbers indicating the lower and upper limits of the interval. The value of minimum must be smaller than or equal to the value of maximum. If there is no upper limit for the interval, this is expressed with an asterisk *. The class `Person` in Figure 4.6 contains an attribute `address: String [1..*]`. This denotes that a person has at least one and possibly multiple addresses. If minimum and maximum are identical, you do not have to specify the minimum and the two dots. For example, [5] means that an attribute adopts exactly *five* values. The expression [*] is equivalent to [0..*]. If you do not specify a multiplicity for an attribute, the value 1 is assumed as default, which specifies a single-valued attribute. The valid notation for multiplicities is summarized in Figure 4.7.

Figure 4.7
Syntax of the multiplicity specification



If an attribute can adopt multiple values, it makes sense to specify whether the attribute is:

- A set (no fixed order of elements, no duplicates)
- A multi-set (no fixed order of elements, duplicates possible)
- An ordered set (fixed order, no duplicates)
- A list (fixed order, duplicates possible)

*Unique, non-unique,
ordered, unordered*

You can make this specification by combining the properties {non-unique} and {unique}, which define whether duplicates are permitted or not permitted, and {ordered} and {unordered}, which force or cancel a fixed order of the attribute values. For example, the attribute `address: String [1..*] {unique, ordered}` contains all the addresses for a person (see Fig. 4.6). As each address should only be contained once, the attribute is labeled {unique}. By specifying {ordered}, we express that the order of the addresses is important. For example, the first address could be interpreted as the main residence.

4.2.4 Operations

Operation *Operations* are characterized by their name, their parameters, and the type of their return value (see Fig. 4.8). When an operation is called in a program, the behavior assigned to this operation is executed. In programming languages, an operation corresponds to a method declaration

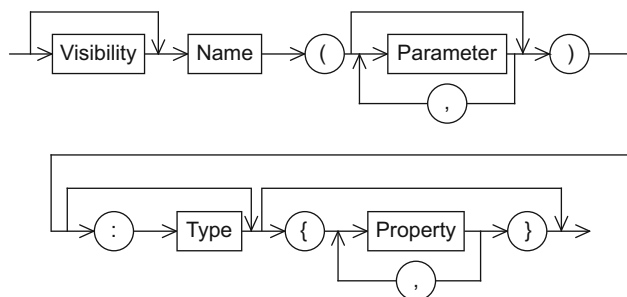


Figure 4.8
Syntax of the operation
specification

or function declaration which is defined but not implemented. The class diagram is not suitable for describing the behavior of objects in detail as it only models signatures of the operations that the objects provide; it does not model how these operations are actually implemented. UML offers special behavior diagrams for depicting the implementation of operations, for example the activity diagram (see Chapter 7).

In a class diagram, the operation name is followed by a list of *parameters* in parentheses. The list itself may be empty. A parameter is depicted similarly to an attribute. The only obligatory information is the name of the parameter. The addition of a type, a multiplicity, a default value, and further properties, such as ordered, unique, or their negated counterparts is optional (see Fig. 4.9).

The optional return value of an operation is specified with the type of the return value. In Figure 4.6, the class *Person* has an operation `updateLastName(newName: String): boolean`. The only parameter, `newName`, has the type `String` and specifies the new name for a person. The return value has the type `boolean`. If *true* is returned, the renaming was successful, otherwise *false* is returned.

If required, you can also prepend a direction to the parameter name. This direction can have one of the following values: *in*, *out*, or *inout* (see Fig. 4.9). The value indicates whether the parameter is an *input parameter*, an *output parameter*, or both. If a parameter has the direction *in*, this indicates that when the operation is used, a value is expected from this parameter. The specification of the direction *out* expresses that after the execution of the operation, the parameter has adopted a new value. If an operation should have multiple return values rather than just one, you can express this using multiple parameters with the direction *out*. The specification of *inout* indicates a combined input/output parameter. If no direction is specified, *in* is the default value. In Figure 4.6, the operation `getName(out fn: String, out ln: String)` has two parameters with the direction value *out*. For example, if we use the operation `getName` in a program by calling `getName(firstName, lastName)`, whereby

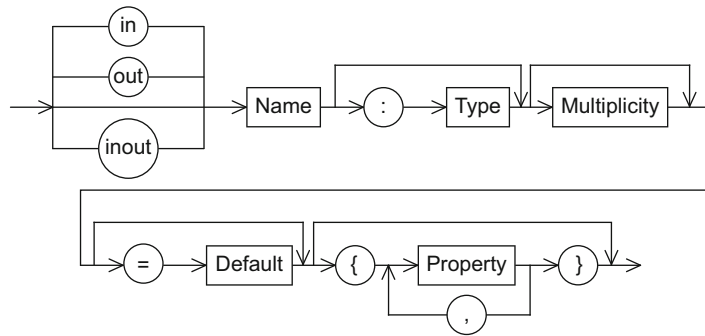
Parameters

Return value

*Input and output
parameters*

`firstName` and `lastName` are variables in the sense of an imperative programming language, successful execution of the operation produces the following results: the variable `firstName` contains the first name and the variable `lastName` contains the last name of the object of type `Person` on which the operation `getName` was called.

Figure 4.9
Syntax of the parameter
specification



4.2.5 Visibility Markers

Visibility The *visibility* of attributes and operations specifies who is and who is not permitted to access them. If an attribute or operation does not have a visibility specified, no default visibility is assumed. Table 4.1 lists the types of visibilities and their meaning in UML. Only an object itself knows the values of attributes that are marked as private. In contrast, anyone can view attributes marked as public. Access to protected attributes is reserved for the class itself and its subclasses. If a class has a package attribute, only classes that are in the same package as this class may access this attribute. Accordingly, the visibility of an operation specifies who is permitted to use the functionality of the operation. Examples are given in Figure 4.4(c) on page 53. Note that the meaning of visibilities can vary in different programming and modeling languages even if they have the same name in the different languages.

Information hiding

Visibilities are used to realize *information hiding*, an important concept in computing. Marking the attributes that represent the state of an object as private protects this state against unauthorized access. Access is therefore only possible via a clearly defined interface, such as via operations that are declared public.

In some cases, class diagrams contain only those attributes and operations that are visible externally. Attributes and operations of classes

that are marked as private are often omitted, as they are important for the realization, that is, the implementation of a class, but not for its use. Therefore, whether or not attributes and operations marked as private are specified depends on the intention behind and the time of creation of the class diagram.

Name	Symbol	Description
public	+	Access by objects of any classes permitted
private	−	Access only within the object itself permitted
protected	#	Access by objects of the same class and its subclasses permitted
package	~	Access by objects whose classes are in the same package permitted

Table 4.1
Visibilities

4.2.6 Class Variables and Class Operations

Attributes are usually defined at instance level. If, for example, a class is realized in a programming language, memory is reserved for every attribute of an object when it is created. Such attributes are also referred to as *instance variables* or *instance attributes*. In Figure 4.10 for example, `lastName` and `dob` are instance variables. If, in an object-oriented program generated from this class of diagram, `person1` is an instance of the class `Person`, for example, `person1.lastName` can be used to refer to the last name of the person. Access to this person’s date of birth is not possible as the visibility of the attribute `dob` is private. To find out the date of birth of `person1`, the function `person1.getDob()` must be called. An operation such as `getDob()` can only be executed if a corresponding instance that offers this operation was created beforehand. In our case, this is the instance `person1`. An operation may use all visible instance variables.

- Synonyms:
- *Instance variable*
 - *Instance attribute*

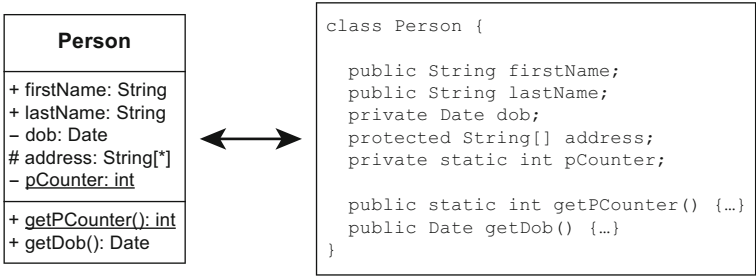
In contrast to instance variables, *class variables* are created only once for a class rather than separately for every instance of this class. These variables are also referred to as *static attributes* or *class attributes*. Counters for the number of instances of a class (see Fig. 4.10) or constants such as π are often realized as static attributes. In the class diagram, static attributes are underlined, just like *static operations*. Static operations, also called *class operations*, can be used if no instance of the corresponding class was created. Examples of static operations are mathematical functions such as `sin(x)` or constructors. Constructors are special functions called to create a new instance of a class. The method invocation `Person.getPCounter()` uses the static opera-

- Synonyms:
- *Class variable*
 - *Class attribute*
 - *Static attribute*

- Synonyms:
- *Class operation*
 - *Static operation*

tion `getPCounter()` defined in Figure 4.10; the operation is called directly via the class and not via an instance. Unless stated otherwise, attributes and operations denote instance attributes and instance operations in most object-oriented languages. We also follow this convention in this book.

Figure 4.10
Translation of a class from
UML to Java

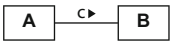


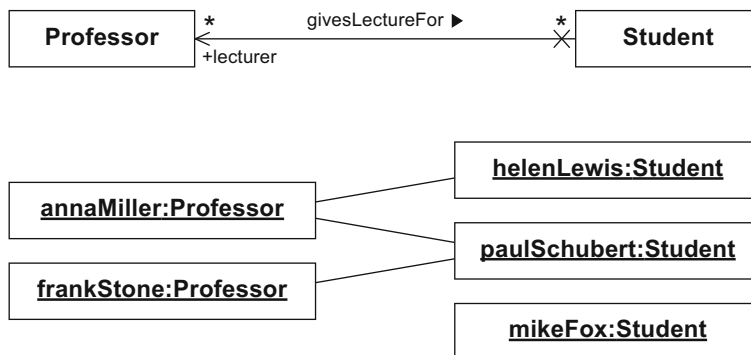
4.3 Associations

Association *Associations* between classes model possible relationships, known as *links*, between instances of the classes. They describe which classes are potential communication partners. If their attributes and operations have the corresponding visibilities, the communication partners can access each other’s attributes and operations. A class diagram can be viewed as a graph in which the classes represent the nodes and the associations represent the edges. Figure 4.11 depicts a class diagram and a valid object diagram. The class diagram shows that the classes Professor and Student are related via the association *givesLectureFor*. In the role as a lecturer, a professor has zero or more students and one student has zero or more professors in the role of lecturer. The object diagram models a concrete scenario.

4.3.1 Binary Associations

Binary association A *binary association* allows us to associate the instances of two classes with one another. The relationships are shown as edges (solid line) between the partner classes involved. The edge can be labeled with the *name of the association* optionally followed by the *reading direction*, a small, black triangle. The reading direction is directed towards one end



**Figure 4.11**

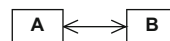
Example of a binary association in a class diagram and a valid object diagram

of the association and merely indicates in which direction the reader of the diagram should “read” the association name. We have already seen a binary association with reading direction in Figure 4.11. In this diagram, the reading direction indicates that professors give lectures for students and not the other way around.

If the edge is directed, that is, at least one of the two ends has an open arrowhead, navigation from an object to its partner object is possible. In simple terms, *navigability* indicates that an object knows its partner objects and can therefore access their visible attributes and operations. The navigation direction has nothing to do with the reading direction, as the example in Figure 4.11 shows. The reading direction indicates that professors give lectures for students. However, the navigability specified indicates that students can access the visible characteristics of professors whose lectures they attend. In contrast, a professor cannot access the visible characteristics of the students who attend the professor’s lecture because the professor does not know them.

A non-navigable association end is indicated by the explicit specification of an X at the association end concerned. For example, if such an X appears at the association end of A for an association between the classes A and B, this means that B cannot access the attributes and operations of A—not even the public ones. Bidirectional edges without arrowheads or X at their ends do not provide any information about the navigation direction but in practice, bidirectional navigability is usually assumed. The navigation direction represents a hint for the subsequent implementation because in object-oriented programming languages, associations are realized as references to the associated objects. An association can also be represented in this way in the class diagram, that is, as an attribute with the appropriate multiplicity, whereby the type of the attribute is the class of the corresponding partner objects. This representation has the same semantics as a navigable association end. Figure 4.12

Navigability



Non-navigability

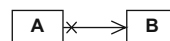
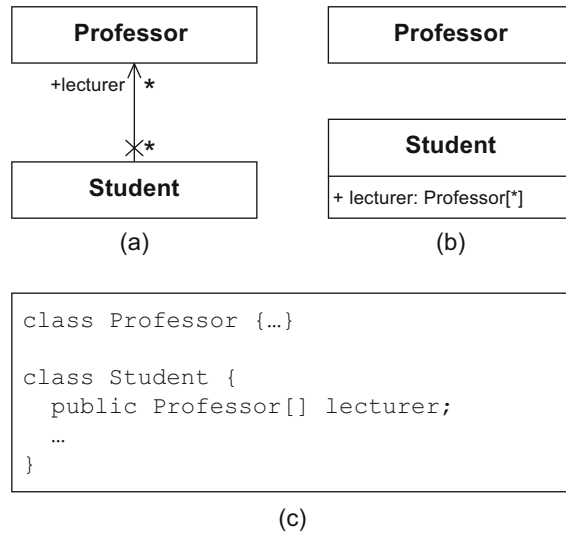
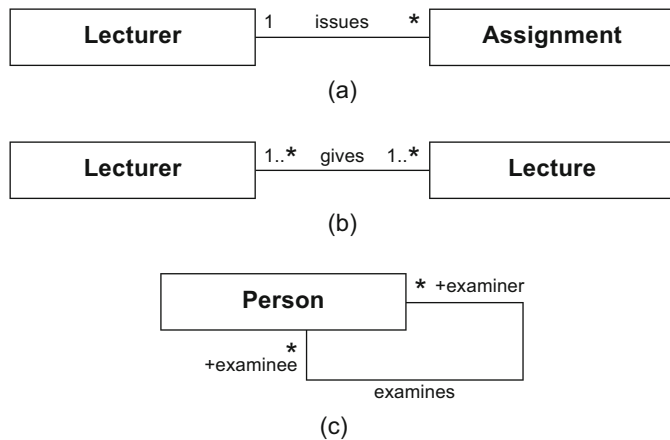


Figure 4.12
Associations in UML and
Java



shows (a) a class diagram in which the student-professor relationship is modeled explicitly as an association, (b) a class diagram in which the relationship is represented by an attribute in the class Student, and (c) the translation into Java. The class diagram in [Figure 4.12\(a\)](#) is preferable, as here the relationship between the classes is visualized explicitly and it is visible immediately, while in the alternative in [Figure 4.12\(b\)](#), the association between Student and Professor can only be recognized by reading the type information of the attribute lecturer.

Figure 4.13
Examples of multiplicity
specifications in binary
associations



In the same way that multiplicities of attributes and parameters are specified, *multiplicities* of associations are given as an interval in the form minimum..maximum. They specify the number of objects that may be associated with exactly one object of the opposite side. The values that the minimum and maximum may adopt are natural numbers and an asterisk *, which expresses that there is no restriction. If minimum and maximum are identical, one value and the dots can be omitted. Again, 0..* means the same as *. Figure 4.13 shows examples of multiplicity specifications for binary associations. Figure 4.13(a) shows that a lecturer may issue no, one, or multiple assignments and that an assignment is issued by exactly one lecturer. No assignment may exist without an association to a lecturer. Figure 4.13(b) shows that a lecturer gives at least one lecture and a lecture is given by at least one lecturer. Finally, Figure 4.13(c) shows that a person in the role of examiner can examine any number (≥ 0) of persons and a person in the role of examinee can be examined by any number of examiners. In the example in Figure 4.13(c), the model does not exclude the case that persons may examine themselves. If this should be prohibited, additional constraints must be specified.

You may also label the association ends with role names. A *role* describes the way in which an object is involved in an association relationship, that is, what role it plays in the relationship. In the association in Figure 4.13(c), the Person adopts the role of examiner or examinee.

To express that an object of class A is to be associated with an object of class B or an object of class C but not with both, you can specify an *xor constraint* (exclusive or). To indicate that two associations from the same class are mutually exclusive, they can be connected by a dashed line labeled {xor}. For example, an exam can take place either in an office or in a lecture hall but not in both (see Fig. 4.14).

Multiplicity

Role

xor constraint

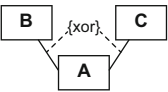
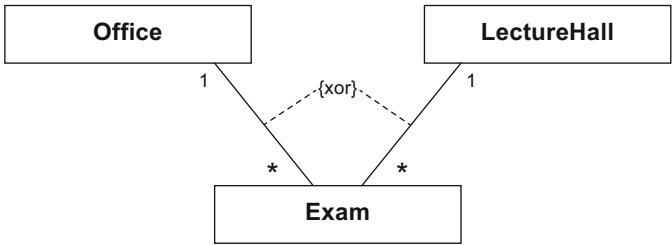
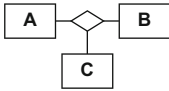


Figure 4.14
Examples of associations with xor constraints

4.3.2 N-Ary Associations

N-ary association



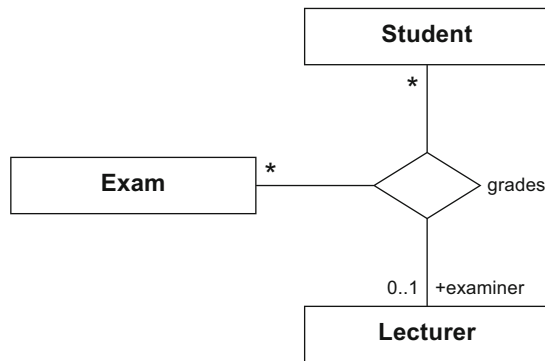
Multiplicities for n-ary associations

If more than two partner objects are involved in a relationship, you can model this using an *n-ary association*. An *n-ary association* is represented with a hollow diamond in the center. The diamond is connected with all partners of the relationship by means of an undirected edge. The name of the association is specified next to the diamond. There are no navigation directions for *n-ary associations*; however, multiplicities and role names are possible. Multiplicities define how many objects of a role/class may be assigned to a fixed $(n - 1)$ -tuple of objects of the other roles/classes.

Figure 4.15 models the relationship grades between the instances of the classes Lecturer, Student, and Exam. The multiplicities are defined as follows: one specific student takes one specific exam with no lecturer (i.e., does not take this exam at all) or with precisely one lecturer. This explains the multiplicity 0..1 for the class Lecturer. One specific exam with one specific lecturer can of course be taken by any number of students and one specific student can be graded by one specific lecturer for any number of exams. In both cases, this is expressed by the multiplicity *. In this model, it is not possible that two or more lecturers grade one student for the same exam.

Figure 4.15

Example of *n-ary* (here ternary) association ...



If you tried to express this ternary association with two binary associations, you would have a model with a different meaning. In the representation shown in Figure 4.16, an exam can be graded by multiple lecturers. The ternary association in Figure 4.15 clearly shows which lecturer a student passed a specific exam with—this is not the case with the diagram shown in Figure 4.16.

For example, with the model shown in Figure 4.15, it is possible to express that student s1 took the exam e1 with lecturer l1 and that student

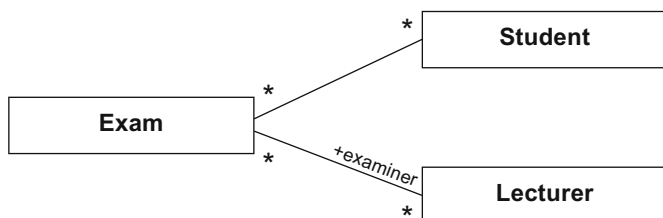


Figure 4.16
... versus example with two
binary associations ...

s2 took the same exam e1 with lecturer l2. With the model shown in [Figure 4.16](#), it is only possible to express that the students s1 and s2 took the exam e1 and that exam e1 has two examiners l1 and l2. With this model, you cannot express which lecturer grades which student.

As an alternative to the ternary association in [Figure 4.15](#), an additional class can be introduced which is connected to the original classes via binary associations (see [Fig. 4.17](#)). However, in this model it is possible that one student is graded multiple times for one and the same exam what is not possible with the model of [Figure 4.15](#).

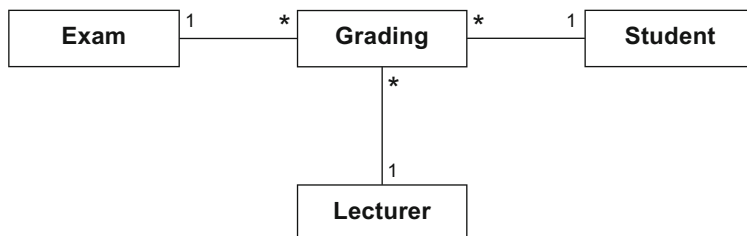


Figure 4.17
... versus example with
additional class

4.4 Association Classes

If you want to assign attributes or operations to the relationship between one or more classes rather than to a class itself, you can do this using an *association class*. An association class is represented by a class and an association that are connected by a dashed line. The association can be binary or n-ary. Although the representation includes multiple components, an association class is *one* language construct that has both the properties of a class and the properties of an association. Therefore, in a diagram, the class and association of an association class must have the same name, although you do not have to name both (see the association classes Enrollment and Grade in [Fig. 4.18](#)). An association class

Association class

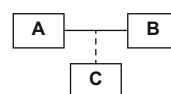
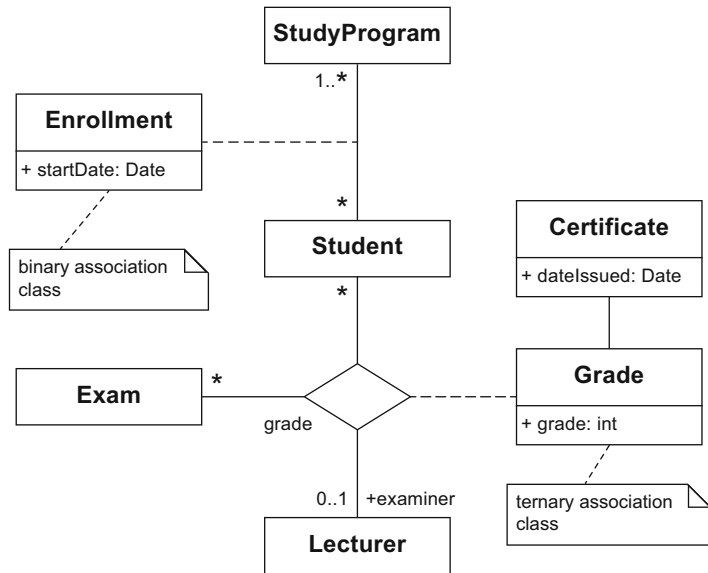


Figure 4.18
Examples of association
classes

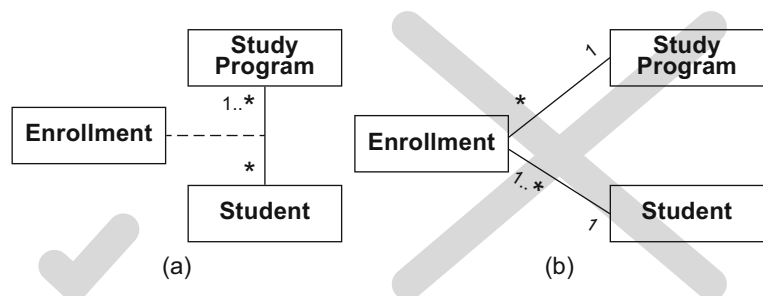


can also have associations with other classes. In [Figure 4.18](#), the association class **Grade**, which contains information about a student's grade for a specific exam, is associated with the class **Certificate**.

In general, you cannot replace an association class with a “normal” class which is itself associated with the original two associated classes, as shown by the following example. Let us assume that we want to model that a student enrolls for at least one study program and has precisely one enrollment for each chosen study program. In turn, any number (≥ 0) of students can enroll for one specific study program. This situation is shown in [Figure 4.19\(a\)](#).

[Figure 4.19\(b\)](#) shows the attempt to model this situation with only “normal” classes. An enrollment is assigned to precisely one student

Figure 4.19
Attempt to model an as-
sociation class with a
“normal” class and cor-
responding relationships



and precisely one study program, while one study program is related to any number of enrollment objects. A student has at least one enrollment. So far the requirements are met. However, if we examine the diagram more closely, we see that in Figure 4.19(b), a student can have multiple enrollments for one and the same study program, which is not the intention. In contrast, in Figure 4.19(a), a student can enroll for a specific study program only once.

If duplicates are explicitly required for an association class, at least one association end must be identified as {non-unique}. If this property is not specified explicitly, the default value {unique} is assumed. In Figure 4.20(a), a student can only be granted an exam meeting to discuss the result of the student's written exam once. Figure 4.20(b) shows a more student-friendly model. There, the use of {non-unique} allows a student to have more than one exam meeting.

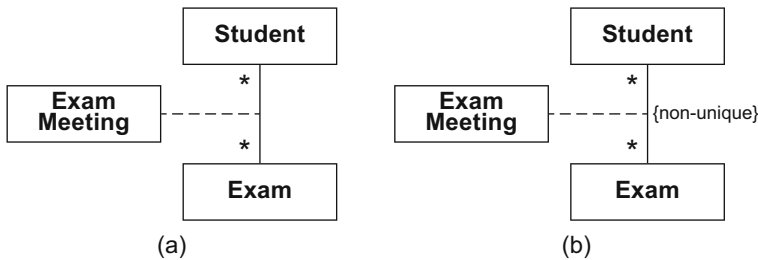


Figure 4.20
Example of {unique} and {non-unique} association ends

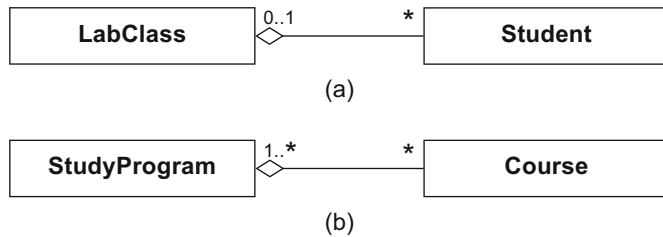
4.5 Aggregations

An aggregation is a special form of association that is used to express that instances of one class are parts of an instance of another class. UML differentiates between two types: *shared aggregation* and *composition*. Both are represented by a diamond at the association end of the class that stands for the “whole”. The differentiation between composition and shared aggregation is indicated by a solid diamond for a composition and a hollow diamond for a shared aggregation. Both are transitive and asymmetric associations. In this case, transitivity means that if B is part of A and C is part of B, C is also part of A. Asymmetry expresses that it is not possible for A to be part of B and B to be part of A simultaneously.

Aggregation

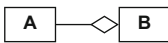
Parts-whole relationship

Figure 4.21
Examples of shared
aggregations



4.5.1 Shared Aggregations

Shared aggregation



In the UML standard, a *shared aggregation* has intentionally informal semantics. In principle, a shared aggregation expresses a weak belonging of the parts to a whole, meaning that parts also exist independently of the whole. The multiplicity at the aggregating end may be greater than 1, meaning that an element can be part of multiple other elements simultaneously. Shared aggregations can therefore span a directed acyclic graph. Figure 4.21 shows two examples of the use of a shared aggregation. In Figure 4.21(a), a lab class consists of any number of students. However, a student can participate in a maximum of one lab class. In Figure 4.21(b), a study program is made up of any (≥ 0) number of courses. A course is assigned to at least one (≥ 1) study program.

4.5.2 Compositions

Composition



The use of a *composition* expresses that a specific part can only be contained in at most one composite object at one specific point in time. This results in a maximum multiplicity of 1 at the aggregating end. The composite objects therefore form a forest of trees, indicating an existence dependency between the composite object and its parts; if the composite object is deleted, its parts are also deleted. Figure 4.22 shows examples of compositions. A lecture hall is part of a building. Due to the multiplicity 1, there is an existence dependency between elements of these two classes. The lecture hall cannot exist without the building. If the building no longer exists, the lecture hall also does not exist anymore. The situation is different for a beamer which is also associated with a lecture hall by a composition. However, the multiplicity 0..1 is specified at the aggregating end. This means that the beamer can exist without the lecture hall, that is, it can be removed from the lecture hall. If the beamer is located in the lecture hall and the lecture hall ceases



Figure 4.22
Examples of compositions

to exist—for example, because the building is torn down—the beamer also ceases to exist. However, if it was removed from the lecture hall beforehand, it continues to exist.

A shared aggregation is differentiated from an association only by the fact that it explicitly visualizes a “part of” relationship. In a composition, the existence dependency signifies a far stronger bond between the composite object and its parts, which means that a composition and an association are not interchangeable. A composition is usually used if the parts are physically embedded in the composite object or are only visible for the composite object. If the parts are referenced externally, this can indicate that a shared aggregation is sufficient. Furthermore, if the composite object is deleted or copied, its parts are also deleted or copied when a composition is used.

*Existence dependency
of a composite object's
parts*

4.6 Generalizations

Different classes often have common characteristics. For example, in [Figure 4.23](#), the classes Student, ResearchAssociate, and AdministrativeEmployee all have the attributes name, address, dob, and ssNo. Students and employees of both types are distinguished by further characteristics specific to the respective class: a student has a matriculation

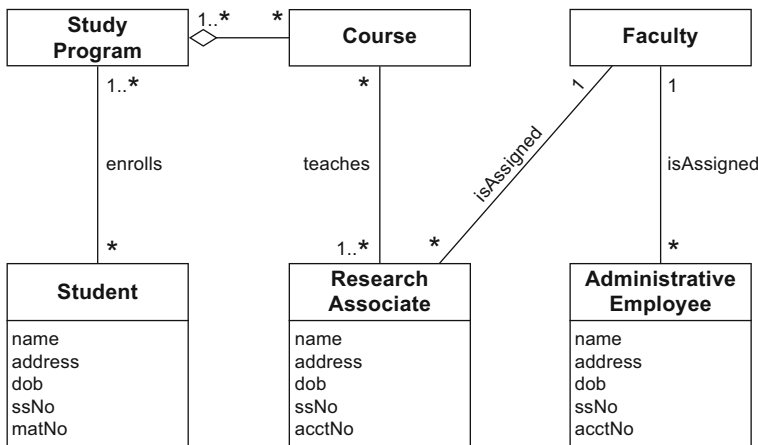


Figure 4.23
Class diagram without
generalization

number and has enrolled for at least one study program; employees have a checking account and are assigned to a faculty. Instances of the class `ResearchAssociate` are in a `teaches` relationship with any number of instances of the class `Course`.

We can use a generalization relationship to highlight commonalities between classes, meaning that we no longer have to define these common characteristics multiple times. Conversely, we can use the generalization to derive more specific classes from existing classes. If we want to add a class `Professor`, which is a subclass of `ResearchAssociate`, in [Figure 4.23](#), we use the generalization to avoid having to copy the characteristics of the class `ResearchAssociate` to the class `Professor`.

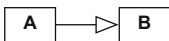
4.6.1 Inheritance

Inheritance from superclass to subclass

Synonyms:

- *Inheritance*
- *Generalization*
- *"Is a" relationship*

Generalization notation



The *generalization relationship* expresses that the characteristics (attributes and operations) and associations that are specified for a general class (*superclass*) are passed on to its *subclasses*. Therefore, the generalization relationship is also referred to as *inheritance*. This means that every instance of a subclass is simultaneously an indirect instance of the superclass. The subclass “possesses” all instance attributes and class attributes and all instance operations and class operations of the superclass provided these have not been marked with the visibility `private`. The subclass may also have further attributes and operations or enter into other relationships independently of its superclass. Accordingly, operations that originate from the subclass or the superclass can be executed directly on the instance of a subclass.

A generalization relationship is represented by an arrow with a hollow, triangular arrowhead from the subclass to the superclass, for example from `Student` to `Person` in [Fig. 4.24](#). The name of a superclass must be selected such that it represents an umbrella term for the names of its subclasses. To ensure that there are no direct instances of the class `Person`, we label this class with the keyword `{abstract}`. The class `Person` therefore becomes an *abstract class* and only its non-abstract subclasses can be instantiated. We will look at details of abstract classes in [Section 4.7](#) on page 72.

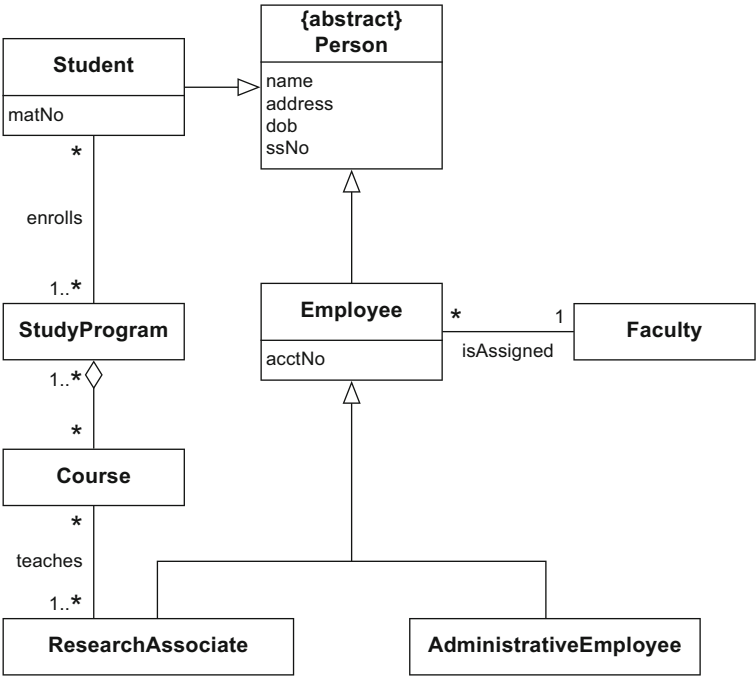


Figure 4.24
Class diagram with
generalization

The generalization relationship is also referred to as an “is a” relationship. For example, every student is a person (see Fig. 4.24). Every research associate and every administrative employee is an employee and, due to the transitivity of the generalization relationship, every administrative employee is also a person. If, as in object-oriented programming languages, we consider a class to be a type, subclasses and superclasses are equivalent to subtypes and supertypes.

UML allows *multiple inheritance*, meaning that a class may have multiple superclasses. For example, a tutor is both an employee of the university and a student (see Fig. 4.25). Due to the transitivity of inheritance, single inheritance creates an inheritance hierarchy, whereas multiple inheritance creates a (directed acyclic) inheritance graph.

*Transitivity of the
generalization
relationship*

*Subtype and supertype
equivalent to
subclass and superclass*

Multiple inheritance

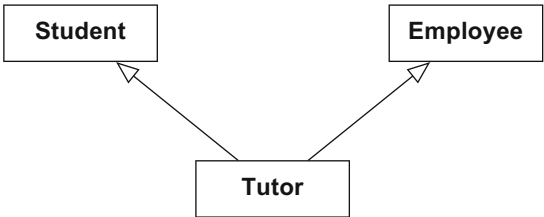


Figure 4.25
Example of multiple
inheritance

4.6.2 Classification

Multiple classification

Classification refers to the “instanceOf” relationship between an object and its class. In many object-oriented programming languages, an object can usually only be the direct instance of precisely one class. In contrast, UML allows *multiple classification*. With multiple classification, an object can be an instance of multiple classes without these classes having to be associated with one another in an inheritance relationship. In contrast to multiple inheritance, no new class inheriting the characteristics of the superclasses involved is introduced.

For example, instances of *Employee* can be differentiated according to their job, that is, whether they are researchers or administrators, and whether they are financed directly via the university or via a project. Multiple classification means that an object can be an instance of multiple classes whose characteristics the object then has. In [Figure 4.26](#), we have divided the generalization relationships into two groups. The sets *Job* and *Financing* form *generalization sets* which group subclasses according to multiple independent criteria. Generalization sets can be described more precisely by the following constraints:

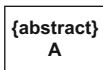
Generalization set

- *Overlapping* or *disjoint*: in an overlapping generalization set, an object may be an instance of multiple subclasses simultaneously. In a disjoint generalization set, an object may be an instance of a maximum of one subclass.
- *Complete* or *incomplete*: in a complete generalization set, each instance of the superclass must be an instance of at least one of the subclasses. In incomplete generalization sets, this is not necessary.

This results in four combinations: {complete, overlapping}, {incomplete, overlapping}, {complete, disjoint}, and {incomplete, disjoint}. If none of these constraints are specified explicitly, {incomplete, disjoint} is the default value. Examples are shown in [Figure 4.26](#): an employee must belong to either the research or administrative personnel but not both. The employee can be financed directly via the university, via a project, via both, or in another, unspecified way, for example via a scholarship.

4.7 Abstract Classes vs. Interfaces

Abstract class



Classes that cannot be instantiated themselves are modeled as *abstract classes*. These are classes for which there are no objects—only their subclasses can be instantiated. Abstract classes are used exclusively to highlight common characteristics of their subclasses and are therefore

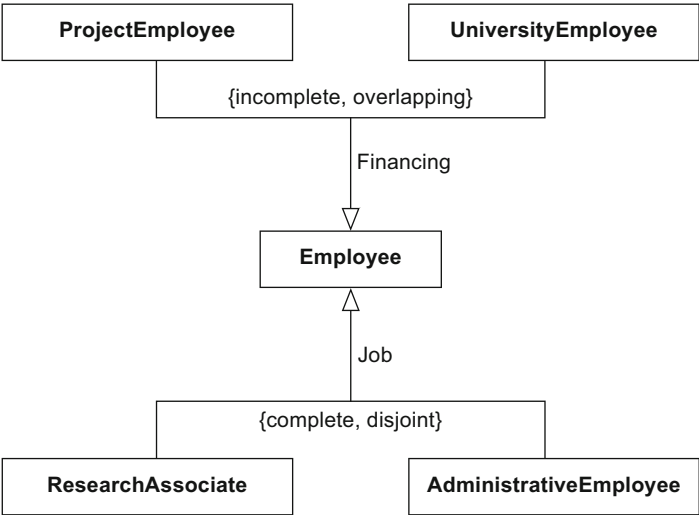


Figure 4.26
Example of multiple classification

only useful in the context of generalization relationships. Operations of abstract classes can also be labeled as abstract. An *abstract operation* does not offer any implementation itself. However, it requires an implementation in the concrete subclasses. Operations that are not abstract pass on their behavior to all subclasses.

Abstract operation

Abstract classes and abstract operations are either written in italic font or indicated by the specification of the keyword {abstract} before their name (see Fig. 4.27). In manually produced class diagrams in particular, the use of the second notation alternative is recommended, as italic handwriting is difficult to recognize.



Figure 4.27
Notation for abstract classes

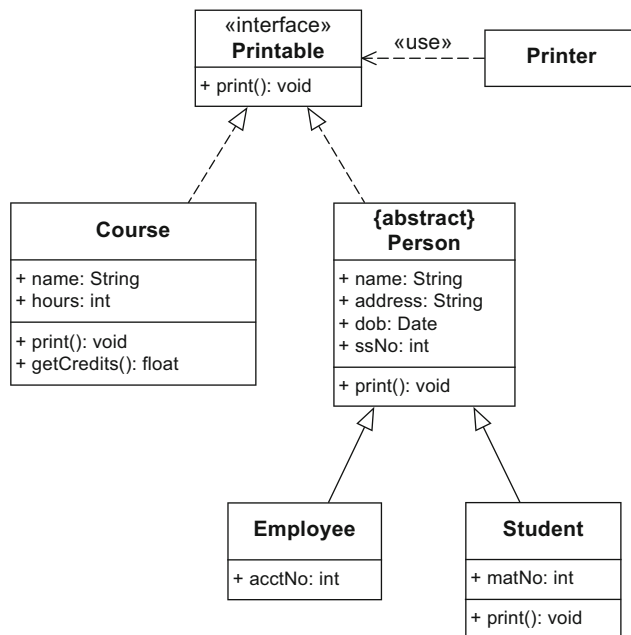
In the example in Figure 4.28, the class Person is abstract. Hence, there cannot be any instances of Person itself but there can be instances of the specific subclasses Employee and Student.

Similarly to the abstract class, an *interface* also does not have an implementation or any direct instances. An interface represents a contract. The classes that enter into this contract, that is, the classes that implement the interface, obligate themselves to provide the behavior specified by the interface. In contrast to the relationship between an abstract class and its subclasses, an “is a” relationship between an interface and the classes that implement it is not necessary. Operations of interfaces never have an implementation.

Interface

An interface is denoted like a class but with the additional keyword «interface» before the name. A dashed inheritance arrow with a hollow, triangular arrowhead from a class to an interface signifies that this class implements the interface. A dashed arrow with an open head with the keyword «use» expresses that a class uses an interface. Let us look at the example from Figure 4.28. The classes `Person` and `Course` implement the interface `Printable`. The classes that implement `Printable` must provide an operation `print()`. This operation is different for every class. For a course, the name and the number of hours are printed; for a `Person`, the name and address are printed. In the class `Student`, the operation `print()` is specified again. This expresses that the `Student` extends the behavior of the operation `print()` inherited from `Person`. The method `print()` is overwritten, meaning that the matriculation number is also printed. For `Employee` this is not necessary, assuming that the behavior specified for `print()` in `Person` is sufficient. The class `Printer` can now process each class that implements the interface `Printable`. Thus, a specific `print()` can be realized for each class and the class `Printer` remains unchanged.

Figure 4.28
Example of an interface



4.8 Data Types

Attributes, parameters, and return values of operations have a type that specifies which concrete forms they may take. For example, the name of a person has the type `String`. A type can be either a class or a *data type*. Instances of data types are also referred to as their *values*. In contrast to instances of classes (objects), values do not have their own identity. If two values are identical, they cannot be differentiated. For example, let us look at the class `Book`, whose instances are different copies of the book *UML@Classroom*. These copies can be uniquely identified and differentiated even though their attributes have the same content. However, different occurrences of a value, for example the number 2, cannot be differentiated. This differentiation becomes evident in the application of the comparison operation `==`, as provided by Java for example. If we compare two variables of the type `int` (integer data type) and both variables have the same value, the result of the comparison operation is true. If we compare two different objects with `==`, the result is false in general even if all attributes have the same values.

In UML, a data type is visualized in the same way as a class, with the difference that the name of the data type is annotated with the additional keyword `«datatype»` (see Fig. 4.29(b)). As the example in Figure 4.29(b) shows, data types can have an internal structure in the form of attributes. In UML, there are also two special forms of data types, namely primitive data types and enumerations.

Primitive data types do not have any internal structure. In UML there are four pre-defined primitive data types: `Boolean`, `Integer`, `UnlimitedNatural`, and `String`. User-defined primitive data types are identified by the specification of the keyword `«primitive»`. Primitive data types may have operations (see Fig. 4.29(a)) that are executed on their values.

Enumerations are data types whose values are defined in a list. The notation is the same as for a class with the specific identification `«enumeration»`. In Figure 4.29(c), the enumeration `AcademicDegree` is defined. This enumeration lists all academic degrees that are known in our system. Therefore, attributes of the type `AcademicDegree` may take the values `bachelor`, `master`, and `phd`. These values are called *literals*.

Class vs. data type

Data type

Primitive data type

Enumeration

Literal

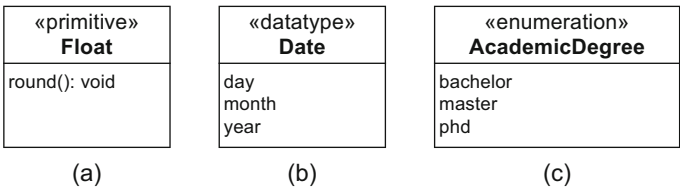


Figure 4.29
Examples of data types

User-defined types are used as specified in the syntax description of attributes and operations in [Figure 4.5](#) (page 54) and in [Figure 4.8](#) (page 57). Let us look at the type definitions from [Figure 4.29](#) again. These could be used in the following attribute definitions: weight: Float, dob: Date, and title: AcademicDegree [*].

4.9 Creating a Class Diagram

UML describes the syntax and semantics of classes and their relationships but not how the classes and relationships are constructed. Unfortunately, it is not possible in principle to completely extract classes and their characteristics from a natural language text automatically. However, there are guidelines for creating a class diagram. Nouns such as person, employee, course, etc. often indicate classes. In contrast, names of values such as *Paul* or *object-oriented modeling* and expressions that indicate the relationships between potential classes are rarely classes. Values of attributes are often expressed by adjectives or also by nouns and operations often result from verbs. The following three aspects are important: which operations can an object of a class execute? Which events, to which the object must be able to react, can theoretically occur? And finally, which other events occur as a result? If the values of an attribute can be derived from another attribute, for example, if the age of a person can be calculated from their date of birth, it should be identified as a derived attribute. Further, it is essential to consider not only the current requirements but also the extensibility of the system.

As we now know how to derive a class diagram from a textual specification, we will do so for the following requirement specification:

Information system of a university

- A university consists of multiple faculties which are composed of various institutes. Each faculty and each institute has a name. An address is known for each institute.
- Each faculty is led by a dean, who is an employee of the university.
- The total number of employees is known. Employees have a social security number, a name, and an e-mail address. There is a distinction between research and administrative personnel.
- Research associates are assigned to at least one institute. The field of study of each research associate is known. Furthermore, research associates can be involved in projects for a certain number of hours, and the name, starting date, and end date of the projects are known. Some research associates teach courses. They are called lecturers.
- Courses have a unique number (ID), a name, and a weekly duration in hours.

1. *Identifying the classes*

First, we must identify the elements that occur in the system University that identify the classes. These are shown in [Figure 4.30](#).

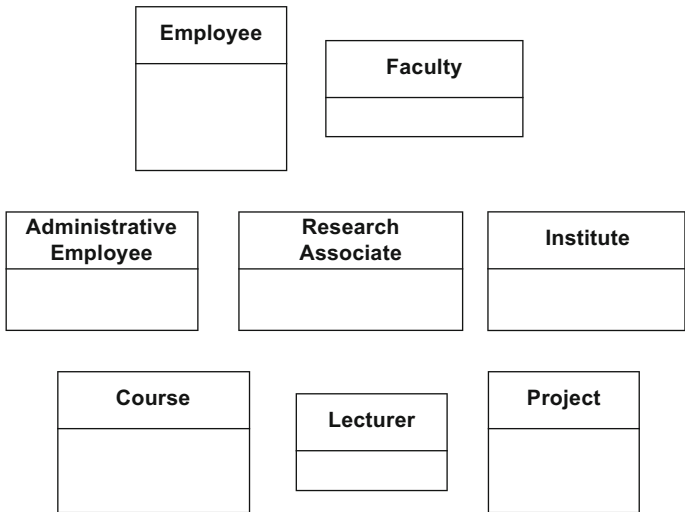


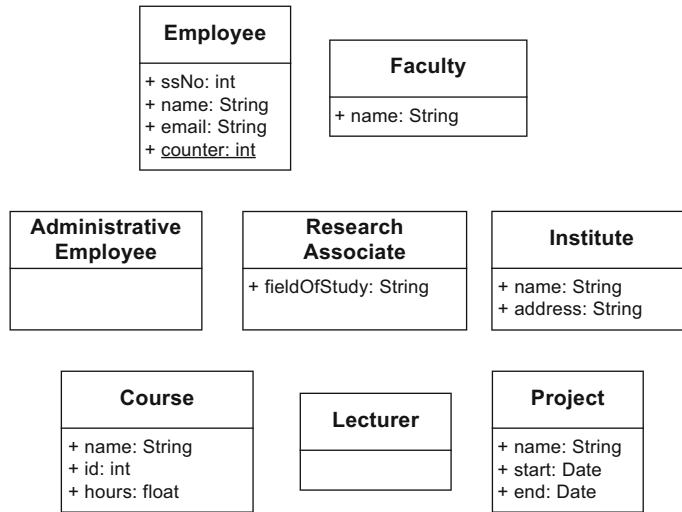
Figure 4.30
Identified classes

As we can see, University is not a separate class. We have not forgotten it—we have intentionally not included it. We are using the class diagram to describe the system University, hence the instance of our model contains those objects that occur within a university, for example, the Vienna University of Technology. If we included a class University which itself consists of other classes from [Figure 4.30](#), we could model multiple university information systems simultaneously. Our model would then also describe, for example, the Johannes Kepler University Linz.

2. *Identifying the attributes*

We can now describe our classes in more detail using attributes. The classes and their attributes are shown in [Figure 4.31](#). We have defined meaningful data types for our attributes even though these are not included in the specification. We also set the visibility of all attributes to public so that in this phase, we do not have to think about which attributes are visible from the outside and which are not. The attribute counter of the class Employee is defined as a class attribute as its values do not belong to an instance. This attribute is increased when an instance of the class Employee is created.

Figure 4.31
Classes and their attributes



3. Identifying the relationships between classes

Classes can be linked with one another in three ways. They can be in a sub-/superclass relationship (generalization), be related by means of an aggregation, or linked via associations.

4.9.1 Generalizations

The following sentences strongly indicate a generalization relationship: “There is a distinction between research and administrative personnel.” and “Some research associates teach courses. Then they are called lecturers.” We model these generalization relationships as shown in [Figure 4.32](#). As every employee of a university belongs to either the research or administrative personnel, we can set the class Employee to abstract.

4.9.2 Associations and Aggregations

To complete the class diagram, we need to add the associations and aggregations and their corresponding multiplicities. The classes Lecturer and Course are linked by means of the association teaches. An employee leads the faculty. Here the employee takes the role of a dean. A faculty

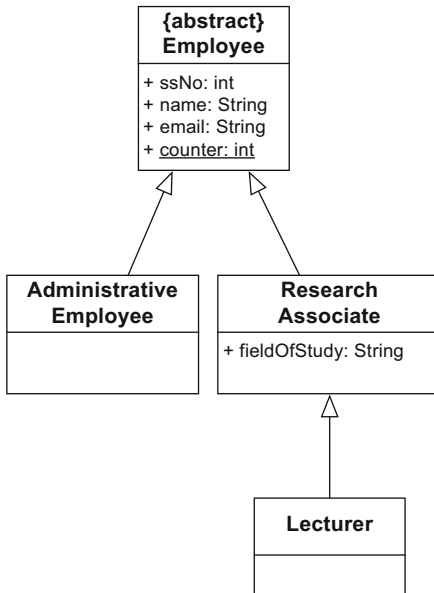
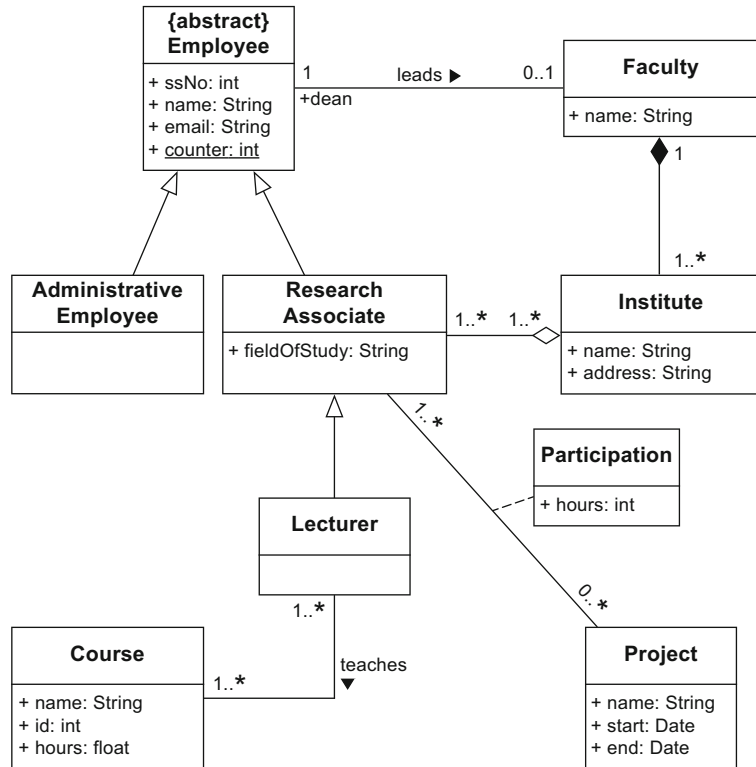


Figure 4.32
Identified generalization
relationships

consists of multiple institutes. We assume that there is an existence dependency which we model with a composition. Research associates are assigned to an institute, meaning they are part of an institute. Using a composition here would be incorrect as there is no existence dependency between instances of *Employee* and *Institute*. However, a shared aggregation is possible in order to represent the parts-whole relationship explicitly. Finally, we have the involvement of research associates in projects, whereby we know the number of hours of participation. For this we need the association class *Participation*. This association class further details the relationship between the project and the research associate with the number of hours. [Figure 4.33](#) shows the complete class diagram for the given task.

Note that the resulting model is not unique even for such small examples; it depends on the one hand on the intended application, and on the other hand on the style of the modeler. For example, if we had created the model with the intention of generating code from it, we would perhaps have designed the interfaces more carefully and specified more differentiated visibilities. It is a matter of taste that *Lecturer* is a separate class but *dean* is a role. We could also have specified *Lecturer* as a role at the end of the association *teaches* which would have been defined between the classes *ResearchAssociate* and *Course*.

Figure 4.33
Class diagram of the information system of a university



4.10 Code Generation

Forward engineering

Class diagrams are often created with the intention of implementing the modeled elements in an object-oriented programming language. As many of the concepts of the class diagram are available in identical or similar form in object-oriented programming languages such as Java, C#, or C++, in many cases a translation can take place automatically and requires only minimal manual intervention. The class diagram is also suitable for documenting existing program code, with the advantage that the relationships between classes are represented graphically. There are a number of tools for reverse engineering program code into class diagrams automatically.

Reverse engineering

Data modeling also involves similar concepts to those of the class diagram. For example, here the entity-relationship diagram (ER diagram) [14] is used—with the exception of different notation, it is very similar to the class diagram. Both diagrams show the elements (classes or entities) of a system and the relationships (associations or relations)

between them. In both cases, these elements are characterized by their attributes. Considerable differences are visible if we compare the focus of the two types of diagrams. While the ER diagram describes the elements of a database, the class diagram shows how to implement the modeled system in an object-oriented programming language. Thus, in the ER diagram, we can define key attributes that are required to identify entries in a table. This is not possible directly in a class diagram but it is also not necessary, as each object is identified by a unique object ID. In contrast, the specification of behavior, which is possible in the class diagram through operations, is not supported in the ER diagram. Therefore, the recommendation is to use the diagram type that is best for the problem in question. The following example again illustrates the connection between a class diagram (see Fig. 4.34) and the Java code generated from it (see Fig. 4.35).

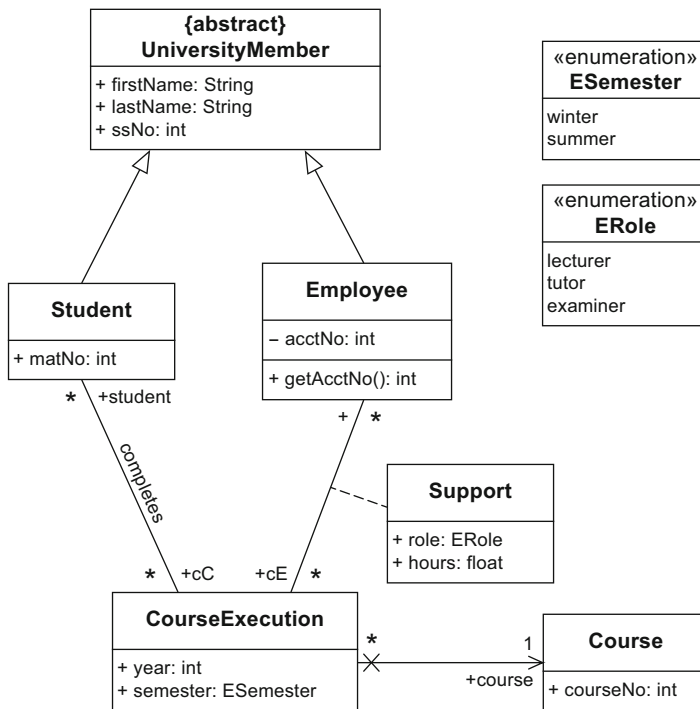


Figure 4.34
Class diagram from which
code is to be generated

Many elements can be translated 1:1. Both abstract and concrete classes are adopted directly in the code with their attributes and operations. In the code, associations are represented as attributes. Note that the multiplicity of an association end is reflected in the type of the at-

tribute. If the multiplicity is greater than one, we can use, for example, an array, as we did for the courses. Instead of arrays we could also use generic data types, for example the Java data type `Collection`; in contrast to arrays, with generic data types we do not have to know the size at initialization [4].

We have to make sure that we implement the navigation directions correctly. The navigation information provided in terms of arrowheads at the association ends tells us which class has to know about which other class—and this is realized via the attributes that model the association ends.

Some concepts, such as association classes or n-ary associations, do not exist directly in common programming languages such as Java. We thus have to consider how to simulate these concepts. Our example contains the association class `Support`. In the code this is implemented as a hash table. A hash table is a data structure that contains elements in the form (key, data). If the key (which must be unique) is known, the related data can be found efficiently.

Up to this point we have been able to describe the structure of elements and their relationships. We were not able to express behavior. In the above example we had only one operation, `getAcctNo()`, which returns the account number of the employee. The content of the method body was generated automatically as it is a getter method that encapsulates the access to a variable of the same name. For other operations, for example, operations that were intended to calculate something, the implementation cannot be derived automatically. UML offers other diagrams for modeling behavior and we will introduce these in the following chapters. To complete this chapter, [Table 4.2](#) summarizes the most important concepts of the class and object diagrams.

```

abstract class UniversityMember {
    public String firstName;
    public String lastName;
    public int ssNo;
}

class Student extends UniversityMember {
    public int matNo;
    public CourseExecution [] cC; // completed c.
}

class Employee extends UniversityMember {
    private int acctNo;
    public CourseExecution [] cE; // supported c.
    public int getAcctNo { return acctNo; }
}

class CourseExecution {
    public int year;
    public ESemester semester;
    public Student [] student;
    public Course course;
    public Hashtable support;
        // Key: employee
        // Value: (role, hours)
}

class Course {
    public int courseNo;
}

Enumeration ESemester {
    winter;
    summer;
}

Enumeration ERole {
    lecturer;
    tutor;
    examiner;
}

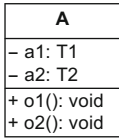
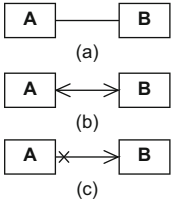
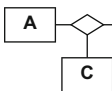
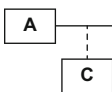
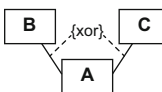
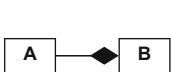
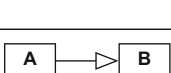
```

Figure 4.35

Java code that can be generated automatically from [Fig. 4.34](#)

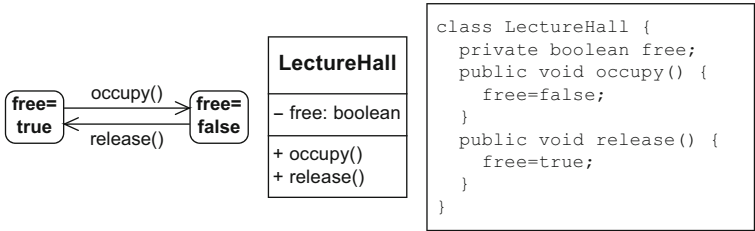
Table 4.2

Notation elements of the class and object diagrams

Name	Notation	Description
Class		Description of the structure and behavior of a set of objects
Abstract class		Class that cannot be instantiated
Association		Relationship between classes: navigability unspecified (a), navigable in both directions (b), not navigable in one direction (c)
N-ary association		Relationship between N (in this case 3) classes
Association class		More detailed description of an association
xor relationship		An object of A is in a relationship with an object of B or with an object of C but not with both
Strong aggregation = composition		Existence-dependent parts-whole relationship (A is part of B ; if B is deleted, related instances of A are also deleted)
Shared aggregation		Parts-whole relationship (A is part of B ; if B is deleted, related instances of A need not be deleted)
Generalization		Inheritance relationship (A inherits from B)
Object		Instance of a class
Link		Relationship between objects

eficial. This figure shows a class LectureHall with an attribute free that can take the values true and false. Calling the operation occupy sets free to false and the lecture hall object changes to the state free=false, which corresponds to the state occupied in Figure 5.1. The events in the state machine diagram are the equivalent of calling the respective operations of the class LectureHall.

Figure 5.2
State machine diagram,
class diagram, and pseu-
docode of a lecture hall



5.1 States and State Transitions



A state machine diagram is a graph with *states* as nodes and *state transitions* as edges. In the diagram, a state is shown as a rectangle with round corners and is labeled with the name of the state. When an object is in a specific state, all internal activities specified in this state can be executed by this object. If internal activities are specified for a state, it is divided into two compartments: the upper compartment of the rectangle contains the name of the state; the lower compartment includes *internal activities*, whereby an activity can consist of multiple actions. We will present the relationship between activities and actions in detail in Chapter 7, which looks at activity diagrams.

Within a state you can model three activities that are executed at a predefined moment. When an activity is specified after the keyword entry, this activity must then be executed when the object enters the state; conversely, the exit activity is executed when the object exits the state. An activity preceded by the keyword do is executed while the object remains in this state, that is, as long as this state is *active*. The respective activity is always specified with a prepended forward slash that clearly identifies it as an activity.

Figure 5.3 shows an extension of the example from Figure 5.1. As long as a lecture hall remains in the state free, that is, as long as the state free is active, the activity Display as available is executed and the lecture hall is displayed in the reservation system. If the lecture hall is occupied,

6.6 Creating a Sequence Diagram

Instead of a final example, in this section we will look at two scenarios that you can use a sequence diagram for. In particular, we will illustrate the connection between the class diagram and the sequence diagram. Then we will conclude the section with a typical application for sequence diagrams, namely the description of design patterns [20].

6.6.1 The Connection between a Class Diagram and a Sequence Diagram

We have repeatedly stated that the different UML diagrams should not be considered independently of one another; they merely offer different views of a certain content. For example, the class diagram shown in [Figure 6.29](#) models a part of a university system that also includes the student administration system.

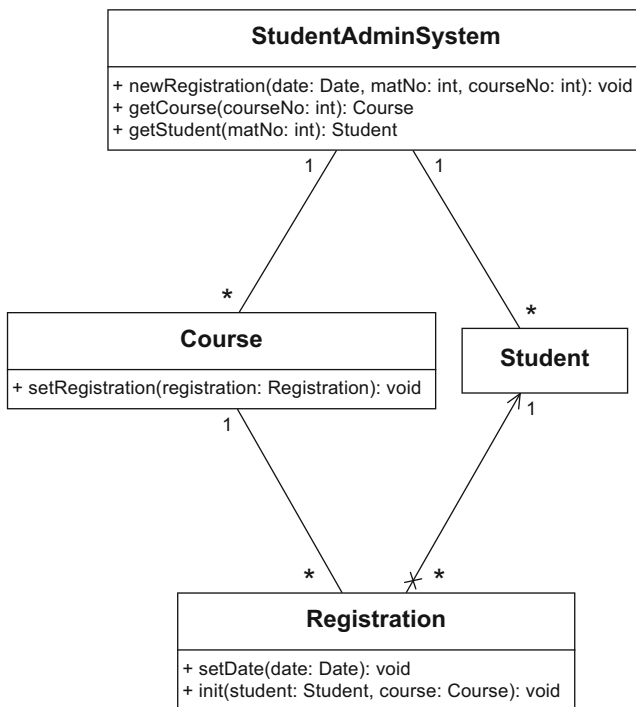
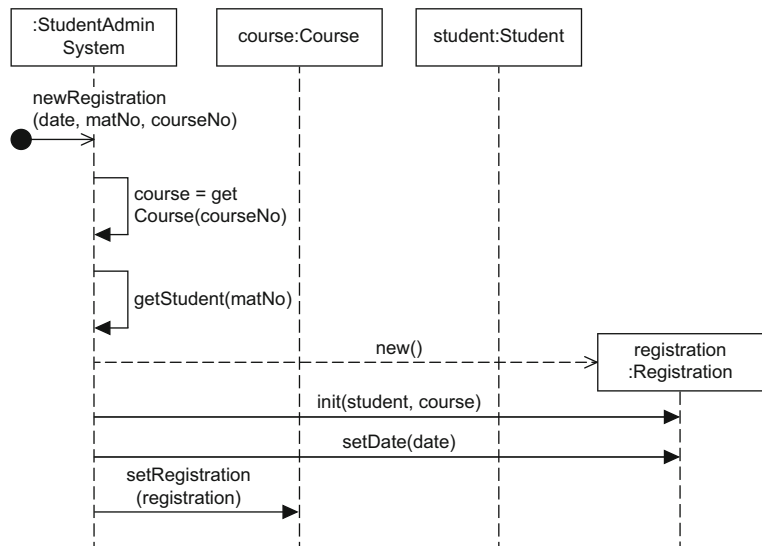


Figure 6.29
Class diagram

The student administration system has direct access to all students and courses. The system knows the registration data for students and courses that is stored in the class `Registration`.

We want to depict the communication that is required to create a new registration of a certain student for a certain course. To do this, the method `newRegistration` of the class `StudentAdminSystem` must be called. To create a new registration, we have to know the student object that belongs to the respective matriculation number and the course object that belongs to the given course number. We can obtain these by calling the operations `getCourse` and `getStudent`. As soon as we have this information, we can create a new object of the type `Registration` and call the `init` operation that sets the student and the course for the registration object. Now we just have to establish the connection between the registration and the course, as navigability in both directions is assumed. We do this by calling the method `setRegistration`. We do not have to do this for the registration and student objects, as navigation from `Student` to `Registration` is not possible. The resulting sequence diagram is shown in [Figure 6.30](#).

Figure 6.30
Sequence diagram based
on class diagram



6.6.2 Describing Design Patterns

Sequence diagrams are often used to describe design patterns. Design patterns offer solutions for describing recurring problems. In the following we will look at the Transfer Object Assembler pattern [18], which describes what happens when a client in a distributed environment requires information from three business objects.

In the solution shown in [Figure 6.31](#), the client requires knowledge about the three business objects in order to access the required data. The client is therefore strongly linked to the business objects—which is generally not desirable. We can use the Transfer Object Assembler pattern to break down these dependencies. In this pattern, an assembler merges the data from multiple business objects into one transfer object that is then transferred to the client. The client thus receives the required data in an encapsulated form. In concrete terms, the pattern is implemented as described below (see [Fig. 6.32](#)).

*Modeling the
communication of the
Transfer Object
Assembler pattern*

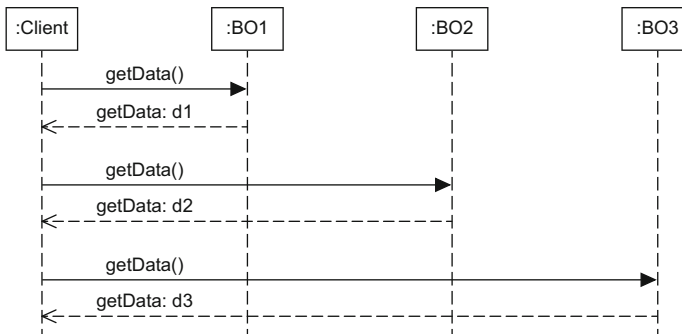


Figure 6.31
Application scenario for
Transfer Object Assembler
pattern

Using `getData()` the client requests the required information from an object of type `TransferObjectAssembler` (TOA). The TOA object creates the object `d` of type `DataTransferObject` (DTO) and fills it with the data from the three different business objects (BO1, BO2, BO3). The data of a business object can be queried using `getData()`. The object `d` offers `setData` methods that allow the entry of data. Finally, the object of type `TransferObjectAssembler` returns `d` to the client.

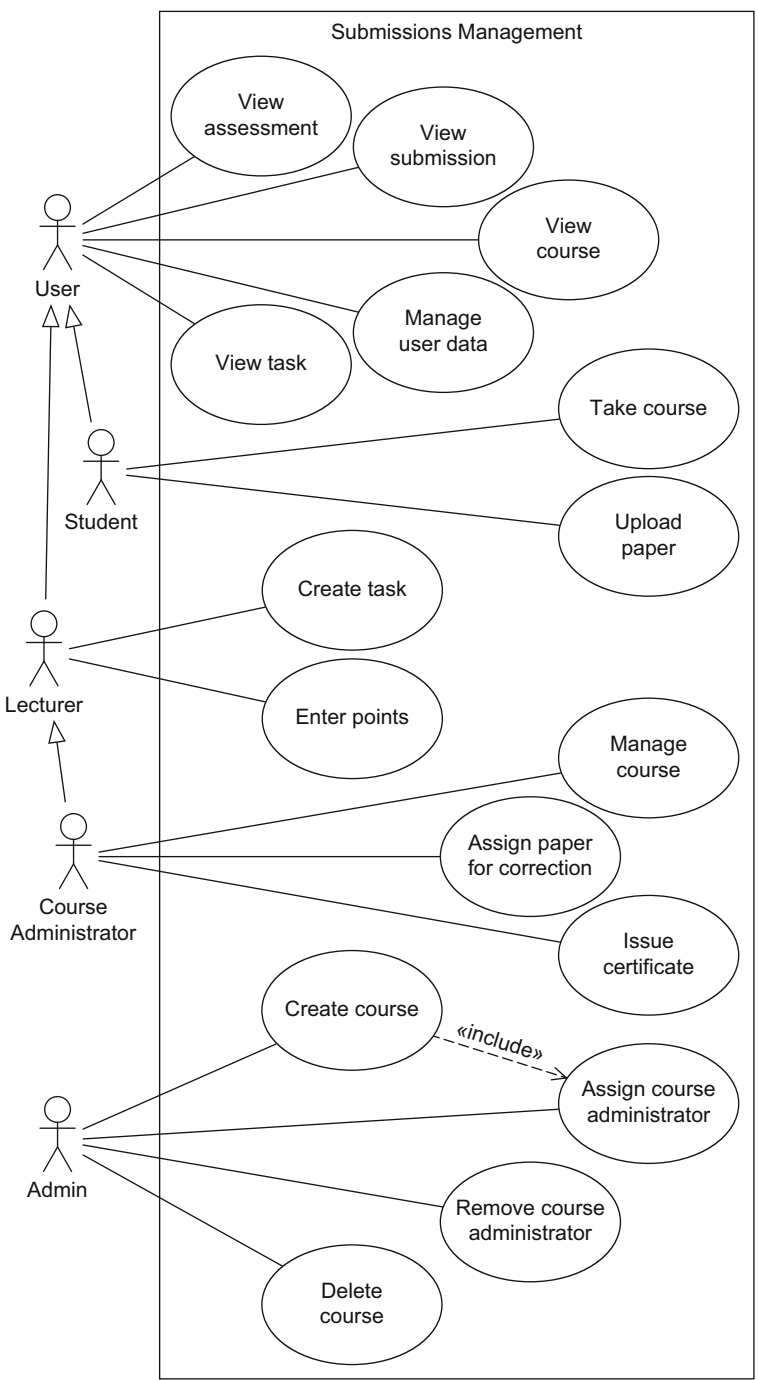
8.2 Example 2: Submission System

UML diagrams are generally used to describe software systems, such as the student administration system of a university, at which we have looked in this book from various aspects. Creating a continuous, detailed model of the entire system which could actually be implemented in executable code would go beyond the scope of this book. However, we will again extract a part of the system to illustrate the interaction of the different diagrams. To this end, we will look at a submission system that is to be used to manage submissions, that is, the students' papers for assignment tasks. The requirements for this system are as follows:

- Every course in the system has lecturers assigned to it. This is done by one of the course administrators, who is also a lecturer. As part of a course, lecturers may create tasks and assess papers submitted by students. Therefore, the lecturers award points and give feedback.
- The course administrator defines which lecturer assesses which papers. At the end of the course, the course administrator also arranges for certificates to be issued. A student's grade is calculated based on the total number of points achieved for the submissions handed in.
- Students can take courses and upload papers.
- All users—students and lecturers—can manage their user data, view the courses and the tasks set for the courses (provided the respective user is involved in the course), and view submitted papers as well as grade points. However, students can only view their own papers and the related grades. Lecturers can only view the papers assigned to them and the grades they have given. The course administrator has access rights for all data.
- A course is created and deleted by an administrator.
- When a course is created, at least one administrator must be assigned to it. Further course administrators can be assigned at a later point in time or assignments to courses can be deleted. The administrator can also delete whole courses.
- Information about users and administrators is automatically transferred from another system. Therefore, functions that allow the creation of user data are not necessary.
- All of the system functions can only be used by persons who are logged in.

The actors and use cases for the specification above are summarized in the use case diagram in [Figure 8.7](#). With regard to the actors, we differentiate between the administrators and all other users, who are in turn subdivided into lecturers and students. With regard to the lecturers, we further differentiate course administrators.

Figure 8.7
Use case diagram for a
submission system



We can derive the use cases directly from the specification above. The use cases are generally not detailed enough for the actual implementation of the system. We now have to highlight exactly which requirements the system must satisfy. However, we do not include this information in the use case diagram as it would make the diagram overloaded and difficult to read. Instead, we document it in the descriptions of the use cases which we learned about in Chapter 3.

The log-in and log-out processes are not represented in the use case diagram as they are not functions desired by the actors but instead contribute to the secure use of the system. Logging in can be considered as a precondition for using the system.

Now that the requirements for the system have been specified, we can zoom into the system and model its structure and behavior. We represent the internal structure of the submission system with the class diagram shown in [Figure 8.8](#). You will notice that all of the actors that appear in the use case diagram are also modeled in the class diagram, even though we stated that they are not part of the system. It is important to understand that in the class diagram, it is the data of the actors that is represented and not the actors themselves. This data is necessary to implement authorizations, the assignment of submissions to students, etc. Information about the users is stored in the class `User`. We use the attribute `authorization` to differentiate between administrators (value `admin`) and all other users (value `standard`). Administrators can be direct instances of the class `User` and lecturers and students are modeled by further classes that have an inheritance relationship to `User`. In principle, this means that it is possible for a lecturer or even a student to be an administrator as well. At first glance, this contradicts our use case diagram, in which the actor `Admin` is in an inheritance relationship to `User`. However, if we consider that the actors in the use case diagram represent roles, our class diagram is correct. One person can of course take multiple roles. In the use case diagram, we have only excluded, for example, that an administrator can automatically view information from the courses. If we really wanted to model a strict differentiation here, we would have to introduce a separate class `Admin` or formulate a constraint that forbids the attribute `authorization` from taking the value `admin` for lecturers or students.

In our class diagram, a lecturer becomes a course administrator by being in a `gives` relationship to a course. Tasks are always assigned to a course, in the same way that submissions are assigned to a task. We model a student's participation in a course using an association class that contains information about the student's total number of points and the grade. Both values are calculated automatically and the attributes are therefore labeled as derived attributes.

The class diagram in [Figure 8.8](#) does not ensure that a lecturer can only assess tasks of a course in which this lecturer is involved. We have to specify these and other restrictions that are important for the consistency of the overall system. Therefore, additional constraints are required. These can be specified in languages such as the Object Constraint Language (OCL) [36] which is beyond the scope of this book. Then the check that determines whether the instances of a model comply with the specified restrictions can often be performed automatically.

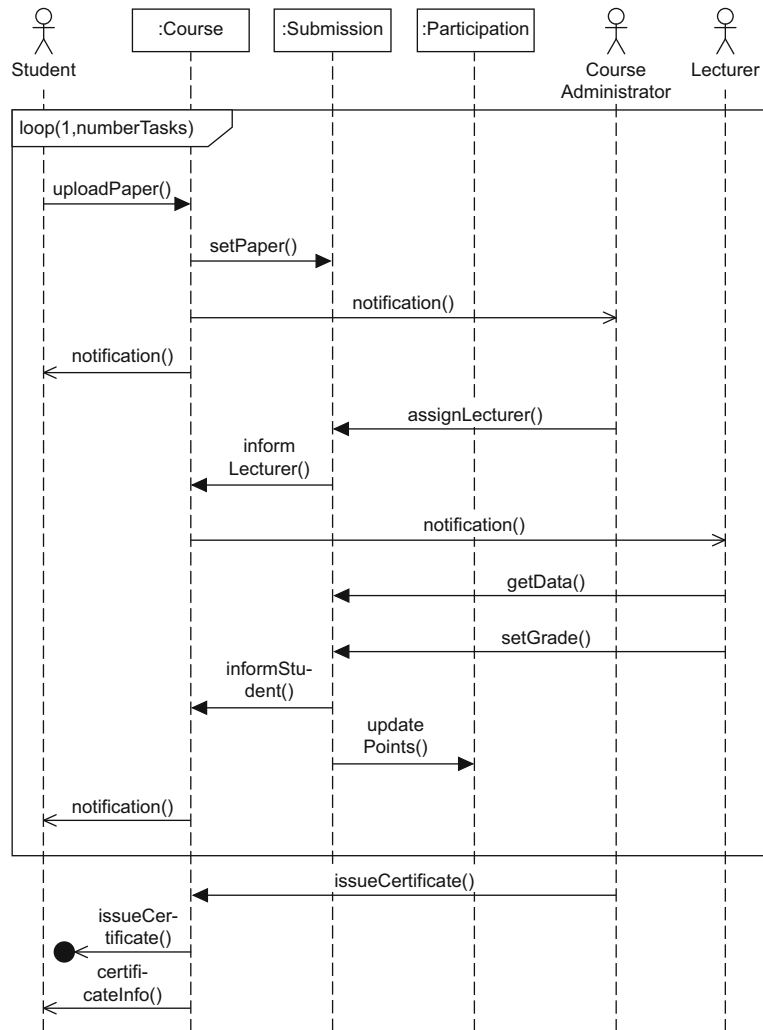
In its current form, the class diagram does not contain any platform-specific information, which means that we have not defined what the implementation should look like. For the general specification of the system to be developed, we want to remain at this abstract level in the subsequent steps.

In the next step, we want to model a typical usage scenario of the submission system, that is, how the actors, the system, and the functions that we have specified in the use case diagram interact in a specific situation. We could create an activity diagram to do this. However, as we want to focus on the communication aspect, we use a sequence diagram instead. [Figure 8.9](#) shows the following usage scenario: A student uploads the solved assignment paper to the submission system. The system informs the course administrator that a new assignment paper has been submitted and confirms to the student that the paper has been successfully received. In the sequence diagram, we do not show the action necessary for saving the paper submitted as it is not relevant for the representation of this specific communication process. Via the submission system, the course administrator assigns a lecturer to the paper. Once the system has informed the lecturer that a paper has been assigned, the lecturer assesses the paper. To do this, the lecturer downloads the paper from the submission system and enters the grade in the system. Then the student is informed that the uploaded paper has been graded. The described communication flow takes place not just once but for every task that has to be completed for a course. Therefore, in the sequence diagram in [Figure 8.9](#), the messages described above are enclosed by a loop fragment.

Once all of the tasks have been processed, the course administrator can arrange for the certificates to be issued. The submission system also informs the student of the final grade.

The sequence diagram described in [Figure 8.9](#) reflects the use of the submission system at a very high abstraction level. Although this highlights the general function of the system, many details are still not specified. Therefore, we have to zoom into the system further. To illustrate this, let us look at the activity Issue certificate shown in [Figure 8.10](#). Here we have decided to use an activity diagram to illustrate the detailed process.

Figure 8.9
Communication flows



We assume that on the user interface, the course administrator sees an overview of the assigned courses. Firstly, the course administrator selects the course to issue certificates for. The students who have taken the course are displayed. The course administrator can then select whether to issue certificates for all or only for certain students. In the latter case, the administrator must also specify the students who shall obtain certificates. The grades are calculated, sent to the student office, and each student is informed of the grade. Note that for the practical implementation, it is extremely important to model all possible flows in as much

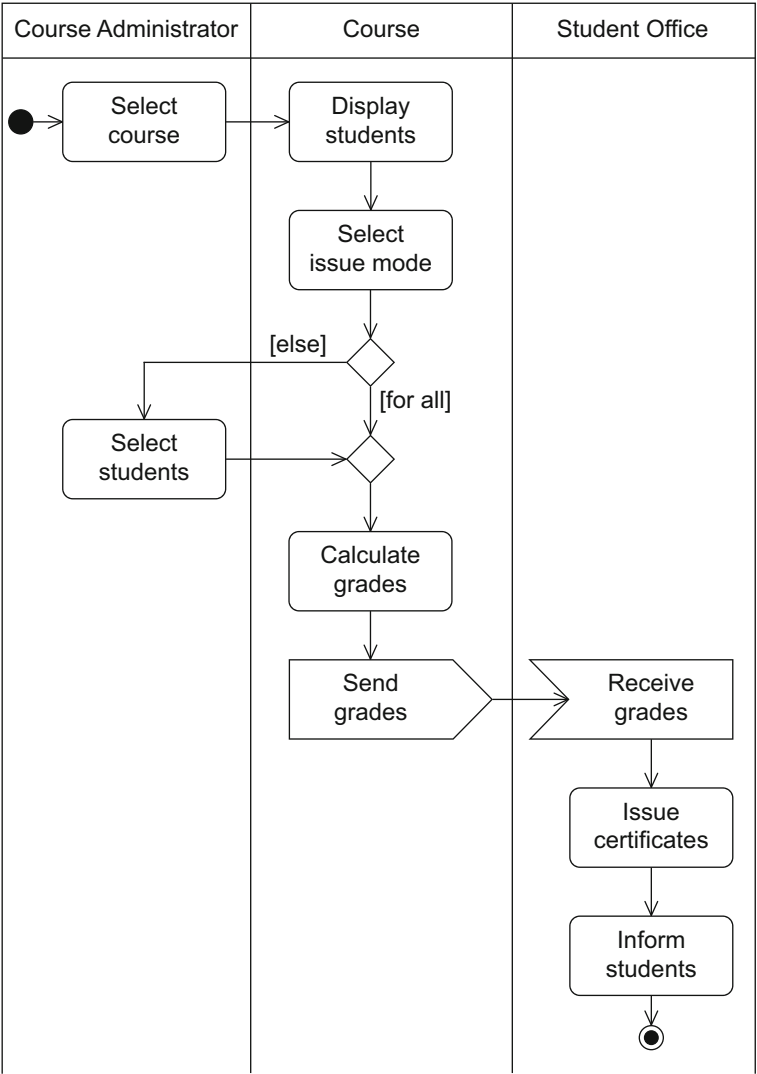
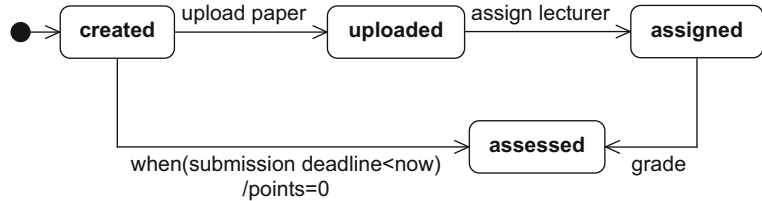


Figure 8.10
Activity diagram for “Issue certificate”

detail as possible and as close to reality as possible. In our activity diagram we have not considered the error situations. It is also not possible in our diagram to manually correct a certificate that has already been issued. All of these cases would also have to be considered. Modeling the processes incorrectly can lead to the system not being accepted by its users. As a result, the increased efficiency expected with the introduction of the system will not be achieved.

To describe the system precisely, it is important also to show the states that the individual components of the system and the system itself can take. As an example, let us look at the state machine diagrams for the submission of a paper (see Fig. 8.11) and for the participation in a course (see Fig. 8.12).

Figure 8.11
State machine diagram for
class Submission



In our system, an instance of class Submission refers to the upload of a paper by a student that then can be assessed by the lecturer. The submission therefore concerns the administration of the file that the student hands over to the lecturer for the assessment, rather than the file itself. When a task is created and released for the students, every student can submit a paper in the system. The submission initially has the state created. It exits this state when the student uploads a paper or when the submission deadline expires—in the latter case the task is assessed with zero points for the student. The submission then changes to the state assessed. If the student submits the solved paper in the system, the submission changes to the state uploaded. It exits this state when the submission is assigned to a lecturer for assessment. Once the assessment has taken place, the submission takes the final state assessed. As the information about a submission is stored for documentation purposes, the submission remains in the state assessed “forever” and no final state is modeled.

The states that a specific participant of a specific course can take are described in a similar way (see Fig. 8.12). In our example, the participation refers to the course participation of a certain student in a certain course. It is documented whether a student has completed a course and, if this is the case, the grade that the student received for this course is saved. When a student has registered for a course, the state of the student’s participation in this specific course is initially not assessed. This clearly shows why we describe the states of course participation and not the states of a student. If a student had the state not assessed, it would not be possible to differentiate between different courses that the student has taken. However, what we want to show is the state of a student with reference to a specific course. In the class diagram, this information is taken into account with the association class Participation (see Fig. 8.8).

For a specific course, a student is initially in the state not assessed, then in the state partially assessed, and finally in the state certificate issued, unless the student is not assessed at all. This can happen, for example, if the student was registered but has never actually attended the course and never completed any activities. The state certificate issued has two substates—positive and negative. Guards specify which of these states occurs. These substates can change if a certificate is corrected.

The diagrams in Figures 8.7 to 8.12 illustrate how the different aspects of a submission system and the interaction between these aspects can be modeled. However, they do not specify any technical details for the implementation, representing instead a sketch that describes how the system should look. Once all of the requirements have been documented in the model, the actual implementation can begin. Different approaches are feasible. One option would be to start the implementation in executable code immediately. However, as no interfaces are specified, sooner or later the different components will not fit together anymore, which makes the maintenance of the system more complicated than with carefully designed interfaces.

Therefore, we recommend refining the model further until you have an exact specification of the system to be developed.

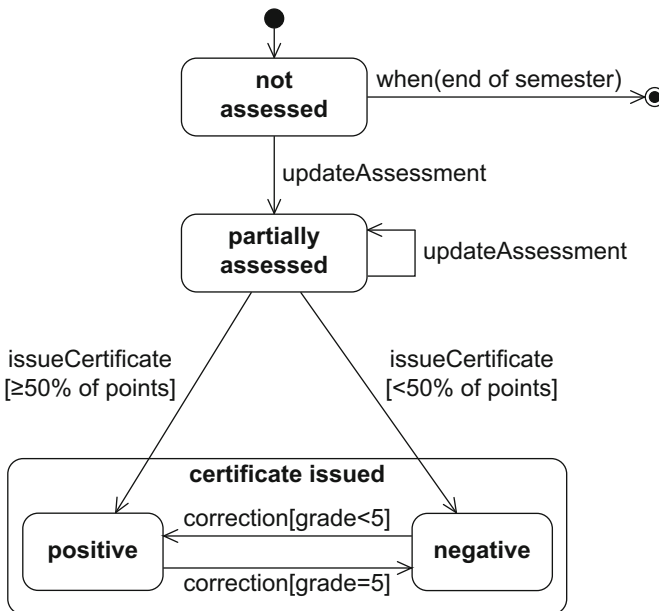


Figure 8.12

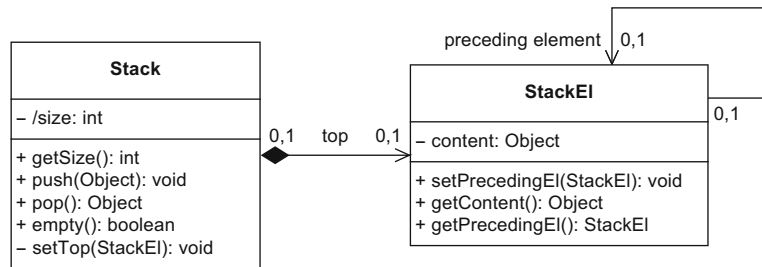
State machine diagram for participation of a student in a course

8.3 Example 3: Data Type Stack

The final example in this chapter is the modeling of the data structure *Stack*. Elements can be placed on the stack using the push function and removed from the stack using the pop function. The order in which elements are removed follows the LIFO principle (Last In, First Out), which means that pop always delivers the element that was last placed on the stack with push and removes it from the stack. Further functions that the class *Stack* should support are the determination of the actual size of the stack, that is, the number of elements on the stack, and the query about whether an element is on the stack at all. As we want to realize a stack with no size restriction, we realize it using a recursive data structure as shown in the class diagram in Figure 8.13. The class *Stack* only knows the uppermost element on the stack. Each element in the stack refers to its direct predecessor. The actual content of a stack element is saved via the private variable *content*.

Figure 8.13

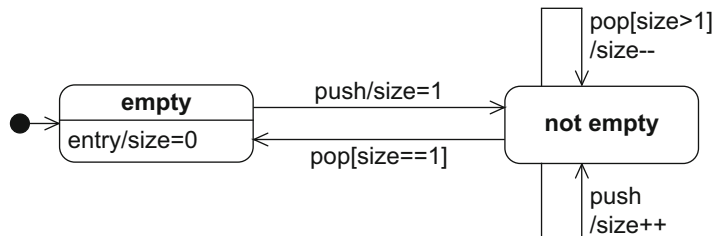
Class diagram for a stack



The states that a stack can take are shown in Figure 8.14. Initially the stack is in the state *empty*. If an element is placed on the stack, the stack changes to the state *not empty*. Every time push is called, the size of the stack increases by one. Every time pop is called, the size reduces by one. If there is only one element on the stack and pop is called, then the stack changes to the state *empty*.

Figure 8.14

State machine diagram for a stack



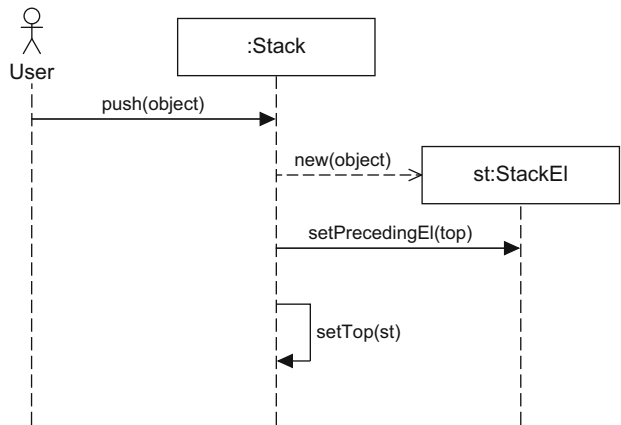


Figure 8.15
Sequence diagram for the addition of an element to the stack

The realization of push and pop is shown in the sequence diagrams 8.15 and 8.16. These diagrams are very close to implementations and reflect how the variables are set.

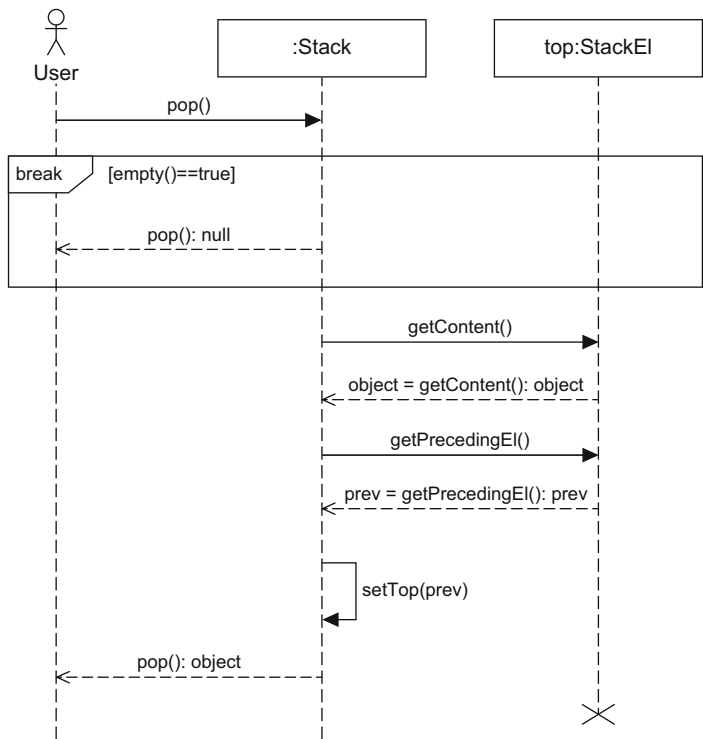


Figure 8.16
Sequence diagram for the removal of an element from the stack

To place an object object on the stack, a new instance of StackEl must be created where the attribute content is set to object. The current top element of the stack becomes the predecessor of the new stack, which now becomes the top element on the stack.

The pop operation reverses the effect of a push operation. The content of the current top element is returned and its predecessor becomes the new uppermost element. If there is no element on the stack, the value null is returned.

8.4 Summary

In the three examples discussed in this chapter, we have not only repeated the most important concepts of UML but have also shown how the different diagrams interact. This interaction allows us to describe a system completely without a developer being supplied with all information at once and being overwhelmed by this flood of information. It enables us to focus on specific questions. The information that is shown in the different diagrams redundantly contributes to making the model more consistent overall, as it allows errors to be found at an earlier development stage and more easily.

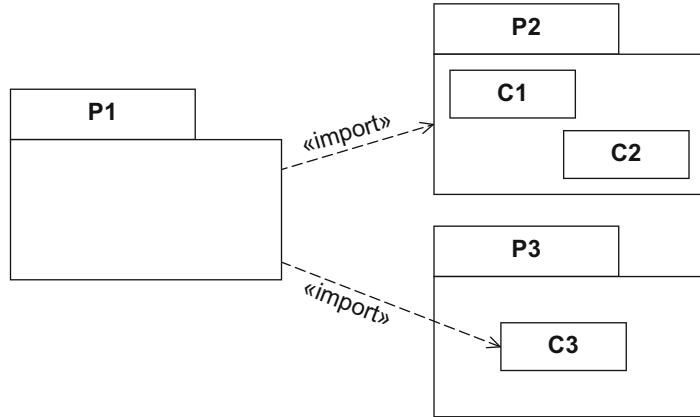
Importing packages

A package can import entire packages in this way. For example, in [Figure 9.2](#), package P1 imports package P2. This makes all visible elements of the imported package visible in the namespace of the importing package. This is handled and noted like every other import relationship.

For a more detailed examination of this topic, see [23].

Figure 9.2

Import relationship



9.2 The UML Metamodel

Metamodel

Critical readers will ask themselves how we know how to apply the concepts of the different diagrams. This is described in the *metamodel*. A metamodel is a model that describes a modeling language. It thus states something *about* other models, a fact which is expressed by the Greek prefix “meta”, which means “about”. In the same way that elements of a model are referred to as model elements, the elements of a metamodel are referred to as metamodel elements.

Superstructure

The UML metamodel, the *superstructure* [35], specifies UML in the form of class diagrams. We use class diagrams to describe which elements a UML diagram may contain and how these elements are applied. However, this also means that a class diagram, which is part of UML, is itself specified by a class diagram. This is comparable with programming languages. Here it is possible and common to write a compiler for C in C, for example. With the concepts that we know from the class diagram, we can now specify modeling languages ourselves. The classes no longer represent persons, courses, and so on but rather language con-

cepts, such as classes, associations, and generalizations, etc. [Figure 9.3](#) shows an example of a simple modeling language that is very similar to the UML class diagram and is itself represented in the form of a class diagram. This metamodel is similar to the metamodel of the real UML class diagram but it is heavily simplified. Almost all classes inherit from the class `NamedElement`. All direct and indirect instances of this class have a name that identifies them uniquely. Associations are described by a separate class. In contrast, a generalization is represented only as a relationship between classes. Of course, we could also model the generalization as a separate class. This would allow us to specify further properties for the generalization, as in “real” UML. For example, in UML, a generalization can be described as disjoint or overlapping (see 4.6.2), which is something we cannot do in our simplified metamodel.

The syntax of UML introduced here is also referred to as *abstract syntax*. If we were to draw an instance of the metamodel from [Figure 9.3](#)—that is, an object model—an association would be depicted as a separate element that connects other elements. An abstract class is then identified by the `isAbstract` flag. This notation is not particularly user-friendly. We have expressed associations simply with a direct connection between the classes that are in a relationship with one another and we have specifically identified abstract classes. This type of notation is more intuitive for human users. Therefore, in addition to the abstract syntax, UML defines the *concrete syntax*, which is a notation optimized for humans.

Abstract syntax

Concrete syntax

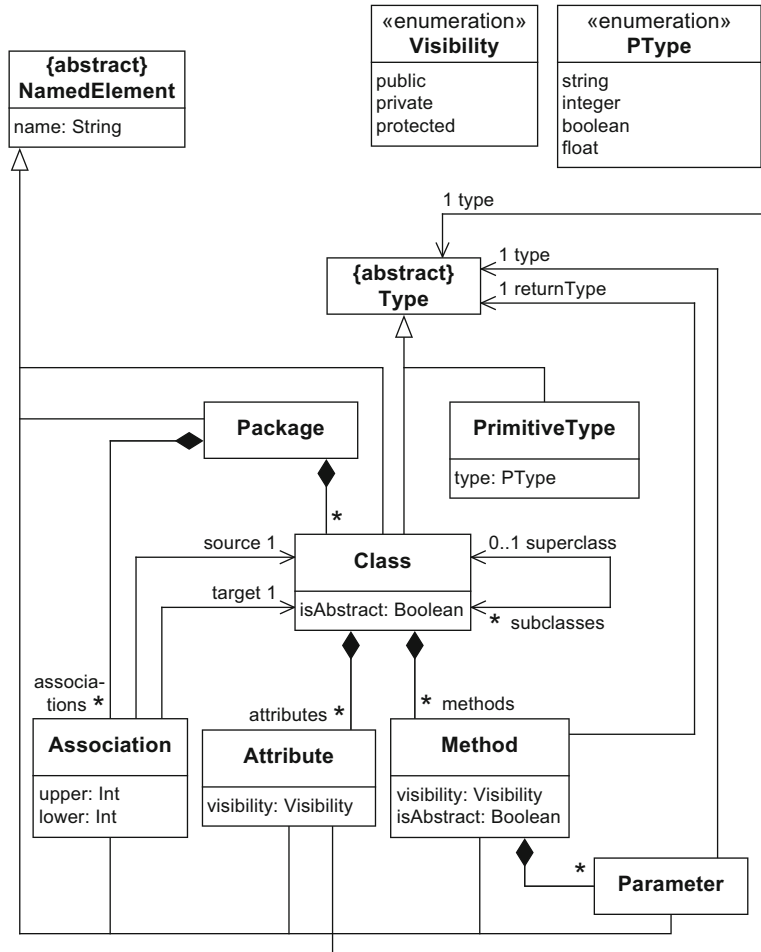
Of course, there are many other important details about metamodels and metamodeling that we could discuss. For example, the obvious question is how the language used to create the metamodel is defined. For this purpose UML has the *infrastructure* [34] which introduces the required concepts. This is the metamodel of UML. We could continue these definitions to infinity but the specification does not go beyond the metamodel level.

Infrastructure

9.3 UML Extension Mechanisms

As a general purpose modeling language, UML provides a stable basis for a wide variety of requirements. It is not defined for specific application domains or for any specific technology. However, in some circumstances, UML is too general and using it involves a considerable amount of effort. In such cases, the use of a language optimized for the given domain and therefore offering special concepts is advantageous.

Figure 9.3
Heavily simplified meta-
model of a class diagram



This type of language can be defined in one of the following three ways:

- Creation of a new metamodel
- Extension and modification of the UML metamodel
- Extension of the UML metamodel with language-inherent mechanisms

*Creation of a new
metamodel*

If the description of the systems to be modeled requires language concepts that are very different to the language concepts of UML, it is probably more practical not to use the language definition of UML and to define your own modeling language by creating a *new metamodel*.

As an alternative to creating a new metamodel, you can also *extend and modify* the UML metamodel in accordance with your requirements. In this case, you introduce new metaclasses and new associations between the metaclasses or overwrite existing properties. This type of extension is also referred to as a heavyweight extension. In many cases, it makes the interoperability of modeling tools more difficult, as of course not all of the tools support the extended metamodel.

Extension and modification of the UML metamodel

UML itself offers *language-inherent extension mechanisms*, that is, extension options that are provided in the language itself. These are already defined at the metamodel level, that is, in the UML infrastructure. The extensions are thus implemented in a controlled way; existing language concepts can only be extended and made more specific and they must not be changed or generalized. This lightweight extension mechanism retains the interoperability between the different modeling tools. These lightweight extensions are based on *stereotypes* and *profiles*, which we will look at more closely below.

Extension of the UML metamodel with language-inherent mechanisms

9.3.1 Stereotypes and Profiles

In Section 9.2, we saw that the UML metamodel itself is also a model. It describes the language elements of UML. The metamodel defines, for example, that a class diagram contains classes, associations, and generalizations, etc. The classes of the metamodel are referred to as metaclasses. A *stereotype* is a special metaclass in the UML metamodel. It allows you to extend any metaclass with additional meta-attributes (tag definitions) and to make it more specific using additional constraints. A metaclass for which a stereotype has been defined remains unchanged. In the simplest case, stereotypes are used to classify metaclasses without introducing additional meta-attributes and constraints.

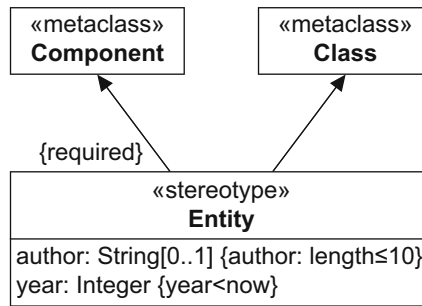
Stereotype

A stereotype is denoted like a class, with the keyword «stereotype» above the name in the first compartment. The second compartment usually contains the meta-attributes. The constraints can be specified either after the meta-attributes or as a note. You can also specify a pictogram for a stereotype. This symbol is used later with the corresponding elements. [Figure 9.4](#) shows an example of a stereotype. The stereotype Entity contains two meta-attributes, author and year, as well as two constraints. One constraint states that the values of the meta-attribute author can have a maximum of ten characters, with the other constraint stating that the value of the meta-attribute year must be smaller than 2006.

A stereotype extends one or more metaclasses. This *extension relationship* is depicted as an arrow with a continuous line and filled arrowhead. The arrow points away from the stereotype to the metaclass. The

Extension relationship

Figure 9.4
Specification of the stereo-
type “Entity”



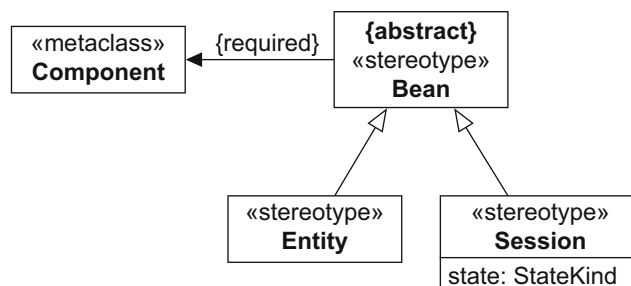
extended metaclass is identified with the keyword «metaclass». Stereotypes defined in this way are optional, meaning that you do not have to use them in the modeling. You can also use model elements that are based on the original definition of the metaclass. To force the use of a stereotype, the extension relationship must be identified as mandatory through the use of the keyword {required}. In Figure 9.4, the stereotype Entity is thus optional for the metaclass Class but mandatory for the metaclass Component.

Stereotypes can be connected to one another via an inheritance relationship. In this context, they can be defined as abstract. A derived stereotype inherits all meta-attributes, constraints, and extension relationships of its higher level stereotypes. Figure 9.5 shows an example of inheritance for stereotypes. The stereotypes Entity and Session are derived from the abstract stereotype Bean. The keyword {abstract} in the name field identifies Bean as abstract. The stereotype Session introduces a new meta-attribute state. The extension relationship between Component and Bean is mandatory, meaning that when a component is modeled, the stereotype Bean must always be used with a specific value.

Profile

Profiles group stereotypes defined for a specific purpose. A profile is a special form of the package that we learned about at the beginning of this chapter. Therefore, a profile has the same notation as a package,

Figure 9.5
Example of inheritance
with stereotypes [35]



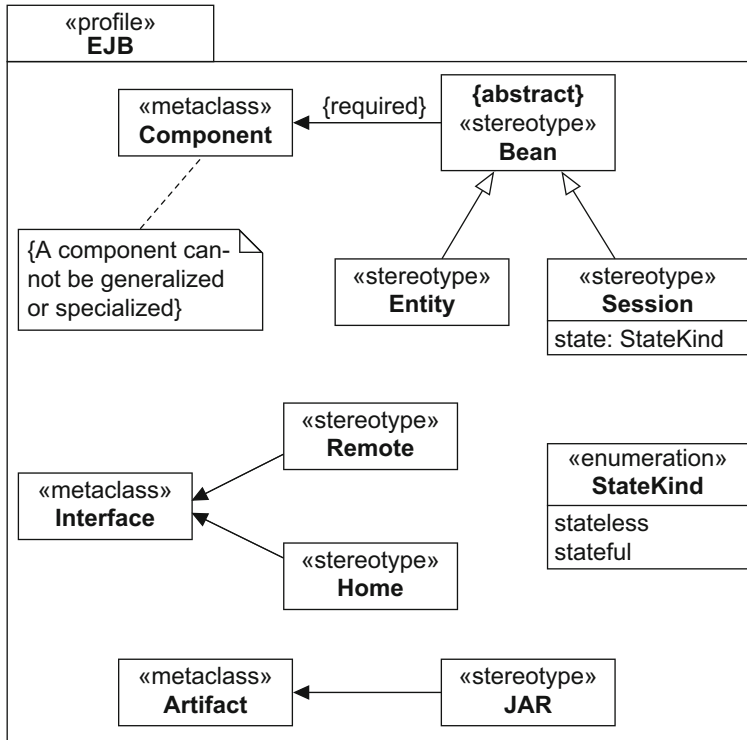


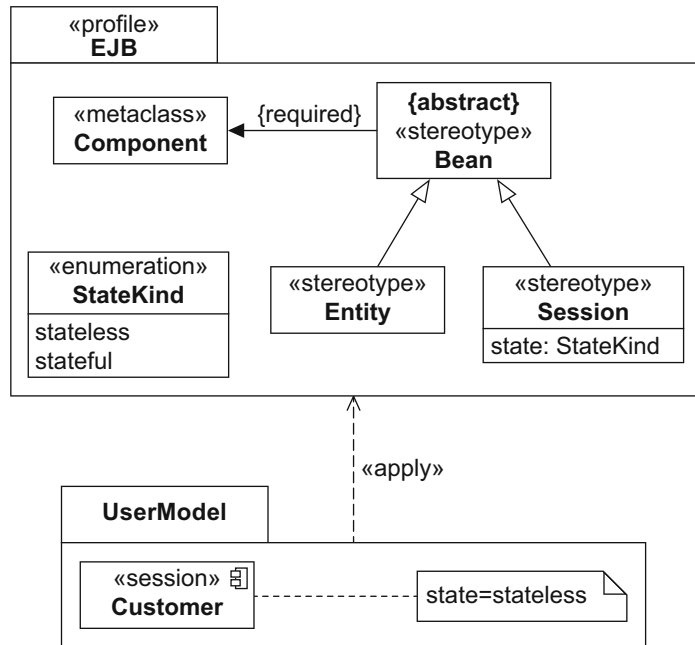
Figure 9.6
Example of a profile [35]

with the keyword «profile» prepended to the profile name. Figure 9.6 shows an example of a profile [35] for Enterprise JavaBeans (EJB).

9.3.2 Applying Stereotypes of a Profile

To use stereotypes in a specific application, you must first integrate the profile that contains the stereotypes. You do this with a dashed arrow with an open arrowhead pointing away from the package of the application towards the profile. This arrow is labeled with the keyword «apply». This imports the stereotypes defined in the profile into the namespace of the package. Figure 9.7 shows an example of the application of a stereotype. The package with the name `UserModel` contains a component `Customer` with the stereotype `Session`. The value for the meta-attribute `state` is specified in a note. The symbol that appears in the upper right corner of `Customer` designates components in UML notation.

Figure 9.7
Example of the application
of a stereotype



9.4 Model-Based Software Development

Models often serve as construction plans for software. For the developers, they stipulate which properties the end product should have. They specify the requirements that a software system must fulfill and describe which components occur in the system and how these components interact. Models are thus the basis for the development of executable code, which is traditionally created by human programmers. If the model and the code are to be kept up-to-date, any changes made must be implemented in both the model and the code. This involves additional effort.

The next step in simplifying the software development process is obvious—namely the automatic generation of the code from the models. This brings us to model-based software development. Models are significantly more than pretty pictures for documenting a software system and for use as sketches or blueprints. In model-based software development, the executable system is created from the models. Here, therefore, models have the same value as code.

*Model-based software
development*

In *model-based software development (MBSD)*, source code is created partially or completely from models. Programming is thus replaced by modeling. The models must describe the function of the system to be developed as precisely as possible. The semantics of the modeling

Index

- absolute time
 - sequence diagram, 130
 - state machine diagram, 94
- abstract
 - abstract actor, **29**, 39
 - abstract class, **70**, 72, 78, 81
 - notation, 73
 - abstract data type, 12
 - abstract operation, 73
 - abstract syntax, 187
 - abstract use case, **32**, 44
- abstraction, 3
- accept event action, **144–145**, 160, 162
- accept time event action, 144
- action, 86, **143–146**, 150, 152, 153
 - accept event, 144–145
 - accept event action, **144**, 160, 162
 - call, 145–146
 - call behavior, 145–146
 - call behavior action, 145
 - call operation action, 146
 - event-based, 144–145
 - exception handling, 159, 160
 - execution permission, 146, 147
 - input parameter, 155
 - output parameter, 155
 - partitioning, 156
 - send signal action, 145
 - time accept event action, 144
- active object, 110
- activity
 - activity diagram, **142**, 143, 144
 - call behavior, 145–146
 - state machine diagram
 - decision node, 90
 - do*, **86**, 87
 - entry*, **86**, 88, 92, 101
 - exit*, **86**, 87, 88, 92
 - history state, 101
 - internal, 86
 - internal transition, 92
 - pseudostate, 89
 - syntax, 88
 - transition, 87, 102
- activity diagram, 20, 138, **141–166**, 169, 175
- activity final node, **151**, 153
- actor
 - activity diagram, **157**, 158, 162
 - sequence diagram, **109**, 175
 - use case diagram, 23, **25–27**, 33, 36, 37, 39, 171, 173
 - abstract, **29**, 39
 - identification, **34–35**, 42
 - inheritance, **28–29**, 34
- Ada, 12
- after*, *see* relative time
- aggregation, **67–69**, 78–79
 - composition, 68–69
 - shared, 68
- alt* operator, 116
- alternative interaction, *see alt* operator
- alternative transition, 90
- anonymous object, 51
- any receive event, 95
- assert* operator, 126
- asserted interaction, *see assert* operator
- association
 - class diagram, 60–65
 - bidirectional, 61

- binary, **60**, 64
- composition, 68–69
- identification, 78
- multiplicity, **62–63**, 68, 81
- n-ary, 64–65
- navigability, 61
- non-unique*, **56**, 57, 67
- ordered*, **56**, 57
- reading direction, 60
- role, **63**, 78, 79
- shared aggregation, 68
- ternary, 64
- unique*, **56**, 57, 67
- unordered*, **56**, 57
- versus aggregation, 69
- xor* constraint, 63
- use case diagram, **27–28**, 34
 - binary, 27
 - error, 39
 - identification, 44
 - multiplicity, 27
- association class, **65–67**, 79, 173
- asynchronous message, 112
- at*, *see* absolute time, sequence diagram
- atomic interaction, *see* critical operator
- attribute, 3, 7
 - class diagram, 51–53, **54–55**, 76
 - access, 61
 - derived, 55
 - multiplicity, **55–56**, 61
 - properties, 55
 - syntax, 54
 - type, 75–76
 - visibility, 58–59
 - identification, 77
 - sequence diagram
 - local attribute, 129–130
- attribute value, 51, 53, 55, 56
- backward engineering, 5
- base use case, **30**, 31
- behavior diagram, 19–21
 - activity diagram, 20, 138, **141–166**, 169, 175
 - communication diagram, 20, **137**
 - interaction diagram, **107**, 136
 - interaction overview diagram, 21, **138**
 - sequence diagram, 20, **107–135**, 175
 - state machine diagram, 19, **85–106**, 132, 169, 177–179
 - timing diagram, 20, **138**
 - use case diagram, 19, **23–47**, 168, 171
- bidirectional navigability, 61
- binary association
 - class diagram, **60**, 64
 - use case diagram, 27
- break* operator, **118**, 119
- C++, 11
- C#, 12
- call behavior action, 145
- call event, 94
- call operation action, 146
- central buffer, **155**, 156
- change event, 94
- class, 6, **52–60**, 70, 75, 76, 173
 - abstract, **70**, 72, 78, 81
 - activity diagram, 142, **146**, 157
 - association class, **65–67**, 79, 173
 - identification, 77
 - inheritance, 69
 - object diagram, 51
 - sequence diagram, 108, 110, 133–134
 - subclass, **70**, 72, 73
 - superclass, **70**
- class attribute, *see* class variable
- class diagram, 17, 26, **49–84**, 113, 133–134, 173–175, 178, 180, 187
 - creation, 76–79
- class hierarchy, 8
- class operation, **59–60**, 70
- class variable, **59–60**, 70
- classification, **72**
- code generation, **80–82**, *see also* implementation
- coercion, 8
- cohesion
 - functional, 184
 - informational cohesion, 184
- combined fragment, **115–126**, 128
 - alt* operator, 116
 - assert* operator, 126
 - break* operator, **118**, 119
 - consider* operator, 125
 - critical* operator, 123–124
 - ignore* operator, 125
 - loop* operator, **118**, 175
 - neg* operator, 126
 - opt* operator, 117
 - par* operator, 121–122
 - seq* operator, 119

strict operator, 120–121
 communication diagram, 20, **137**
complete, 72
 completion event, 88, 95
 completion transition, 88
 complex state, *see* composite state
 component diagram, 18
 composite state, 96–102
 composition, 68–69, *see* strong
 aggregation
 composition structure diagram, 18
 concrete syntax, 187
 concurrency, *see also* parallelism
 activity diagram, 148, **150**, 151, 152
 sequence diagram, 115, **119–124**
 state machine diagram, 91
 concurrent interaction, *see par* operator
 condition, *see also* guard
 activity diagram
 postcondition, 142
 precondition, 142
 sequence diagram, 119
 use case diagram
 extend relationship, 31
 postcondition, **36**, 46
 precondition, **36**, 39, 46, 173
 connector, 147
consider operator, 125
 constructor, **59**, 113
 continuation marker
 activity diagram, *see* connector
 sequence diagram, 126, **129**
 control flow
 activity diagram, 144, **146–153**, 162
 sequence diagram, 110
 control flow edge, 144
 control token, **154**, 161
 coregion, 123
 create message, 113
critical operator, 123–124
 current time, 131

 data flow, *see* object flow
 data flow parameter, *see* stream
 data store, **155**
 data token, 154, 155
 data type, 7, 54, **75–76**, 77
 decision behavior, 149
decision input, *see* decision behavior
 decision node
 activity diagram, **149**, 150, 152, 162
 state machine diagram, 90–91

deep history state, **101**, 102
 default value, 55
 deployment diagram, 18
 descriptive model, 4
 design pattern, 135
 destruction event, 113
 diagram, 15
 behavior diagram, 19–21
 activity diagram, 20, 138, **141–166**,
 169, 175
 communication diagram, 20, **137**
 interaction diagram, **107**, 136
 interaction overview diagram, 21,
 138
 sequence diagram, 20, **107–135**,
 175
 state machine diagram, 19, **85–106**,
 132, 169, 177–179
 timing diagram, 20, **138**
 use case diagram, 19, **23–47**, 168,
 171
 frame, 15
 structure diagram, 17–19
 class diagram, 17, 26, **49–84**, 113,
 133–134, 173–175, 178, 180, 187
 component diagram, 18
 composition structure diagram, 18
 deployment diagram, 18
 object diagram, 17, **49–52**, 60
 package diagram, **18**, 184
 profile diagram, 19
 direct execution, 111
disjoint, 72
do activity, **86**, 87
 duration, *see* time constraint, time
 interval, transmission duration

 edge
 activity diagram, 142, **143**, 147, 152
 connector, 147
 control flow edge, 144
 lightning bolt edge, 160
 object flow edge, 144, 154
 class diagram, *see also* association
 state machine diagram, *see also*
 transition
 effect, 87
 Eiffel, 11
else
 sequence diagram, 116
 state machine diagram, **90**, 95
 encapsulation, 7

- entity-relationship model, 12
- entry* activity, **86**, 88, 92, 101
- entry point, 99–100
- enumeration, 54, **75**
- event
 - activity diagram, 160
 - event-based action, 144–145
 - sequence diagram, 107
 - destruction event, 113
 - receive event, 111
 - send event, 111
 - starting event, 130
 - state machine diagram, 87, 102–104
 - any receive event, 95
 - call event, 94
 - change event, 94
 - completion event, 88, 95
 - signal event, 94
 - time event, 94
 - types, 94–95
 - use case diagram
 - trigger, **24**, 36
- event specification, **110**
- event-based action, 144–145
- exception handler, 159–160
- exception handling
 - interruptible activity region, 160–161
 - sequence diagram, 118, 159–161
- exception interaction, *see break*
- operator
- exclusive or, *see xor* constraint
- execution
 - direct, 111
 - indirect, 111
- execution path, *see trace*, sequence diagram
- execution semantics, **146**, 152, 154, 157
- execution specification, 111
- existence dependency, **68**, 79
- exit* activity, **86**, 87, 88, 92
- exit point, 99–100
- extend*, **30**, 37
 - condition, 31
- extending use case, 31
- extension point, 31
- extension relationship, 189
- external transition, **92**, 96
- final state, **92**
 - composite state, **96**, 104
 - history state, 101
 - orthogonal state, 97
 - submachine, 98
- flow final node, **151**, 152
- fork, *see* parallelization node
- forward engineering, 5
- found message, 114
- fragment, *see* combined fragment
- functional cohesion, 184
- gate, 128
- general purpose modeling language, 14, 187, 193
- generalization, *see* inheritance
- generalization relationship, 73
- generalization set, 72
 - complete*, 72
 - disjoint*, 72
 - incomplete*, 72
 - overlapping*, 72
- genericity, 8
- guard, *see also* condition
 - activity diagram, 144, 146, **149**
 - sequence diagram, 116–118
 - state machine diagram, **87**, 89, 90, 102, 179
- history state, 101–102
 - deep, **101**, 102
 - shallow, 101
- ignore* operator, 125
- implementation, 12
 - class diagram, 59, 61, 73
 - sequence diagram, 126, 132, 181
- import* relationship, 185
- importing packages, 185–186
- in* parameter, 57
- include*, **30**, 32, 34, 37, 44
- included use case, 30
- incomplete*, 72
- indirect execution, 111
- information hiding, 58
- informational cohesion, 184
- infrastructure, *see* metamodel
- inheritance, 7–8
 - class diagram, **70–71**, 78
 - class, 69
 - multiple inheritance, 71
 - use case diagram
 - actor, **28–29**, 34
 - use case, **32**, 34
- initial node, **147**, 150, 152, 169
- initial state, 90

- composite state, **96**, 103
- orthogonal state, 97
- submachine, 98
- inout* parameter, 57
- input parameter, *see* parameter
- instance, 6, **52**, 59, 60, *see also* object
- instance attribute, *see* instance variable
- instance level, 59
- instance operation, **60**, 70
- instance variable, **59**, 70
- inter-object behavior, **107**, 108
- interaction, **107**, 110, 115
 - alternative, *see alt* operator
 - asserted, *see assert* operator
 - atomic, *see critical* operator
 - concurrent, *see par* operator
 - exception, *see break* operator
 - invalid, *see neg* operator
 - irrelevant, *see ignore* operator
 - iterative, *see loop* operator
 - optional, *see opt* operator
 - relevant, *see consider* operator
 - repeated, *see loop* operator
 - sequential with strict order, *see strict* operator
 - sequential with weak order, *see seq* operator
- interaction diagram, **107**, 136
- interaction overview diagram, 21, **138**
- interaction partner, 107, **108–110**, 111
- interaction reference, **127–128**, 138
- interface, 58, **72–74**, 79, 179, 184
- internal activity, 86
- internal transition, **88**, 92, 96
- interruptible activity region, 160–161
- intra-object behavior, 21
- invalid interaction, *see neg* operator
- irrelevant interaction, *see ignore* operator
- is a* relationship, *see* inheritance
- iterative interaction, *see loop* operator
- Java, 12, 52, 60–62, 75, **80–82**
- join, *see* synchronization node
- life cycle of an object, 19, 96, 102
- lifeline, **108–110**, 111, 113, 123, 132, *see also* interaction partner
- link, 51, 60
- list type, *see* enumeration
- local attribute, 129–130
- loop, *see loop* operator
- loop* operator, **118**, 175
- lost*, *see* lost message
- lost message, 114
- MBSD, *see* model-based software development
- merge node, **150**, 152, 162
- message, 107, **110–114**
 - asynchronous, 112
 - between objects, 7
 - create, 113
 - found, 114
 - lost, 114
 - response message, 112
 - synchronous, 112
 - syntax, 113
 - time-consuming, 114
- message exchange, 119
- message with duration, *see* time-consuming message
- metamodel, 13, 14, **186–187**
- model, 2–5
 - abstraction, 4
 - descriptive, 4
 - prescriptive, 4
- model-based software development, 5, **192–194**
- modeling language, 2
 - domain-specific, 2
 - general purpose, 2
 - multiple classification, 72
- multiple inheritance
 - class diagram, 71
- multiplicity
 - class diagram
 - association, **62–63**, 68, 81
 - attribute, **55–56**, 61
 - operation, 57
 - use case diagram, 27
- multivalued role, 109
- n-ary association, 64–65
- namespace, 184
- navigability, 61
 - bidirectional, 61
 - non-navigability, 61
- neg* operator, 126
- nested state, *see* composite state
- new*, *see* object creation
- node
 - activity diagram
 - activity final node, **151**, 153

- central buffer, **155**, 156
- data store, **155**
- decision node, **149**, 150, 152, 162
- flow final node, **151**, 152
- initial node, **147**, 150, 152, 169
- merge node, **150**, 152, 162
- parallelization node, **150**, 152
- synchronization node, **150**, 152
- state machine diagram
 - decision node, **90**
 - parallelization node, **91**, 97
 - synchronization node, **91**, 97
- non-navigability, 61
- non-unique*, **56**, 57, 67
- now*, *see* current time
- object, 6–7
 - class diagram, **52**, 61, 63, 134
 - classification, 72
 - object diagram, **50–52**
 - anonymous object, 51
 - sequence diagram, 108, 109, 113
 - active object, 110
 - creation, 113
 - state machine diagram, 85, 86
 - creation, 90
 - deletion, 92
- Object Constraint Language, **13**, 16, 175
- object creation, 90, 113
- object diagram, 17, **49–52**, 60
- object flow, 144, **154–156**, 162
- object flow edge, 144, 154
- Object Management Group, 13
- Object Modeling Technique, 12
- object node, 154, 155
- object orientation, **6–9**, 11, 29, 52, 61, 71, 80, 113
- Object-Oriented Software Engineering, 12
- OCL, *see* Object Constraint Language
- OMG, *see* Object Management Group
- OMT, *see* Object Modeling Technique
- OOSE, *see* Object-Oriented Software Engineering
- operand, 115
- operation, 7
 - activity diagram, 142
 - call, 146
 - class diagram, 51, 52, 54, **56–58**
 - abstract operation, 73
 - access, 54, 61
 - class operation, **59–60**, 70
 - instance operation, **60**, 70
 - multiplicity, 57
 - syntax, 56
 - visibility, 58–59
 - sequence diagram, 118
 - call, 113
 - state machine diagram
 - call, 94
- operator, *see* combined fragment
- opt* operator, 117
- optional interaction, *see* *opt* operator
- ordered*, **56**, 57
- orthogonal state, 97–98
- out* parameter, 57
- overlapping*, 72
- overloading, **7**, 8
- package, 184–186
 - import, 185–186
- package*, **58–59**, 185
- package diagram, **18**, 184
- par* operator, 121–122
- parallelism, *see also* concurrency
 - sequence diagram, 121
 - state machine diagram, 97
- parallelization node
 - activity diagram, **150**, 152
 - state machine diagram, **91**, 97
- parameter
 - activity diagram, **142**, 143–145, 154, 155
 - class diagram, **57**, 75
 - in* parameter, 57
 - inout* parameter, 57
 - out* parameter, 57
 - sequence diagram, 113, 127, **129–130**
 - state machine diagram, 94
- partition, **156–159**, 162
 - subpartition, 157
- Petri net, 142, 146
- pin, **155**
- polymorphism, 8–9
- postcondition
 - activity diagram, 142
 - use case diagram, **36**, 46
- precondition
 - activity diagram, 142
 - use case diagram, **36**, 39, 46, 173
- prescriptive model, 4
- primitive data type, 75

- private*, 7, **58–59**, 70
- profile diagram, 19
- programming language, 11, 52, 56, 58, 59, 61, 71, 80, 82
- protected*, 7, **58–59**
- pseudostate, 89
- public*, 7, **58–59**, 77
- qualified name, 185
- reading direction, 60
- readOnly*, 55
- receive event, 111
- relative time
 - sequence diagram, 130
 - state machine diagram, 94
- relevant interaction, *see consider*
 - operator
- repeated interaction, *see loop* operator
- requirements engineering, 34
- response message, 112
- return value
 - class diagram, **56**, 75
 - sequence diagram, 113, 128
- role
 - class diagram, **63**, 78, 79
 - sequence diagram, 108
 - multivalued, 109
 - use case diagram, 28
- selector, 109
- self*, 110
- self-transition, 93
- send event, 111
- send signal action, 145
- seq* operator, 119
- sequence diagram, 20, **107–135**, 175
- sequence of events, *see trace*
- sequential interaction with strict order, *see strict* operator
- sequential interaction with weak order, *see seq* operator
- shallow history state, 101
- shared aggregation, 68
- signal
 - activity diagram, 144
 - receive, 147
 - send, 145
 - sequence diagram, 107
 - state machine diagram, 94
- SIMULA, 11
- Smalltalk, 11
- software development process, 14
- software system, 2, 49, 107, 171
- source state, 87
- standard process, **36**, 41
- state, 7, **86–92**
 - composite, 96–102
 - history state, 101–102
 - initial state, 90
 - orthogonal, 97–98
 - pseudostate, 89
 - source state, 87
 - submachine, 98
 - target state, 87
- state invariant, 132
- state machine diagram, 19, **85–106**, 132, 169, 177–179
- state transition, *see transition*
- static attribute, *see class variable*
- static operation, *see class operation*
- stereotype, 189–191
- stream, 154
- strict* operator, 120–121
- structure diagram, 17–19
 - class diagram, 17, 26, **49–84**, 113, 133–134, 173–175, 178, 180, 187
 - component diagram, 18
 - composition structure diagram, 18
 - deployment diagram, 18
 - object diagram, 17, **49–52**, 60
 - package diagram, **18**, 184
 - profile diagram, 19
- subclass, 7, **70**, 72, 73
- submachine, 98
- submachine state, 98
- subpartition, 157
- subtype, 71
- superclass, 8, **70**
- superstructure, *see metamodel*
- supertype, 71
- swimlane, *see partition*
- synchronization node
 - activity diagram, **150**, 152
 - state machine diagram, **91**, 97
- synchronous message, 112
- syntax
 - abstract, 187
 - attribute specification, 54
 - concrete, 187
 - message specification, 113
 - multiplicity specification, 56
 - operation specification, 56

- parameter specification, 58
- transition specification, 88
- system, 2, 23–27, 167, 177, *see also*
 - software system
- behavior, 160
- boundary, **25**, 37
- interaction, 108
- state, 85, 87, 143
- static structure, 49
- target state, 87
- terminate node, 92
- ternary association, 64
- time
 - absolute
 - sequence diagram, 130
 - state machine diagram, 94
 - current, 131
 - relative
 - sequence diagram, 94, 130
- time constraint, 130–131
 - absolute time
 - sequence diagram, 130
 - state machine diagram, 94
 - current time, 131
 - relative time, 130
 - time interval, 131
 - timing mark, 131
 - transmission duration, 131
- time event, 94
 - after*, *see* relative time, state machine diagram
 - when*, *see* absolute time, state machine diagram
- time interval, 131
- time-consuming message, 114
- timing diagram, 20, **138**
- timing mark, 131
- token
 - control token, **154**, 161
 - data token, 154, 155
- token concept, **146**, 147–153, 157, 159, 162
- trace
 - sequence diagram, **111**, 115, 120–122, 124, 125
 - state machine diagram, 102–104
- transition, **86–89**, 90, 96
 - alternative, 90
 - completion transition, 88
 - composite state, 96–98
 - concurrent, 91
 - external, **92**, 96
 - internal, **88**, 92, 96
 - self-transition, 93
 - syntax, 88
- transitivity, 67, **71**
- transmission duration, 131
- trigger event, **24**, 36
- type, **54**, 56, 57, 61, 71
 - data type, 7, 54, **75–76**, 77
 - list type, *see* enumeration
 - primitive data type, 75
 - subtype, 71
 - supertype, 71
- UML, *see* Unified Modeling Language
- Unified Modeling Language, **1**, 4, 13, 14
- unique*, **56**, 57, 67
- unordered*, **56**, 57
- use case, 23, **24–25**, 26, 33
 - abstract, **32**, 44
 - base use case, **30**, 31
 - description, **35–36**, 45
 - extend*, **30**, 37
 - condition, 31
 - extending, 31
 - identification, 43–44
 - include*, **30**, 32, 34, 37, 44
 - included, 30
 - inheritance, **32**, 34
 - relationship with actor, 27
- use case diagram, 19, **23–47**, 168, 171
 - creation, 34–45
 - error, 37–41
- value
 - attribute value, 51, 53, 55, 56
 - default value, 55
- variable
 - class variable, **59–60**, 70
 - instance variable, **59**, 70
- visibility, 7, **58–59**
 - package*, **58–59**, 185
 - private*, 7, **58–59**, 70
 - protected*, 7, **58–59**
 - public*, 7, **58–59**, 77
- when*, *see* absolute time, state machine diagram
- XMI, *see* XML Metadata Interchange Format
- XML Metadata Interchange format, 14
- xor* constraint, 63